

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT THÀNH PHỐ HỒ CHÍ MINH

KHOA ĐIỆN - ĐIỆN TỬ

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

[CHÈN LOGO HCMUTE]

ĐỒ ÁN TỐT NGHIỆP

**XE ROBOT TỰ HÀNH ĐỊNH VỊ BẰNG
ODOMETRY VÀ TÌM ĐƯỜNG A***

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH (ĐẠI TRÀ)

Sinh viên thực hiện: Nguyễn Quốc Việt

MSSV: 22119254

GVHD: Th.S Nguyễn Ngô Lâm

TP. Hồ Chí Minh – tháng 6-2025

LỜI CAM ĐOAN

Người thực hiện Nguyễn Quốc Việt xin cam đoan đề tài này là công trình nghiên cứu của bản thân tôi dựa trên những kiến thức đã được học và tích lũy được sự dẫn dắt, chỉ bảo của ThS. Nguyễn Ngô Lâm. Toàn bộ phần cứng, firmware ESP32 và ứng dụng di động Flutter trong đồ án đều do tôi tự thiết kế, lập trình và kiểm thử. Kết quả có được trong đồ án là hoàn toàn trung thực và không có sự sao chép từ các công trình nghiên cứu trước đó.

Người thực hiện

Nguyễn Quốc Việt

LỜI CẢM ƠN

Để hoàn thành tốt báo cáo đồ án tốt nghiệp của ngành Công nghệ Kỹ thuật Máy tính, trước hết em xin chân thành cảm ơn đến quý Thầy/Cô trong trường Đại học Sư phạm Kỹ thuật Thành phố Hồ Chí Minh nói chung và khoa Điện - Điện tử nói riêng. Đặc biệt là thầy ThS. Nguyễn Ngô Lâm đã tận tình hướng dẫn, giúp đỡ và tạo điều kiện thuận lợi cho em trong suốt quá trình thực hiện đồ án. Em xin gửi đến thầy lời cảm ơn chân thành nhất.

Đồng thời em cũng xin gửi lời cảm ơn đến bạn bè vì đã hỗ trợ, đóng góp ý kiến trong quá trình thực hiện đề tài.

Mặc dù đã cố gắng hết sức nhưng vì kiến thức còn hạn chế, không thể tránh khỏi những thiếu sót. Do đó, em rất mong nhận được sự đóng góp ý kiến từ phía Thầy/Cô để có thể hoàn thiện tốt hơn nữa, đồng thời tích lũy thêm kinh nghiệm để đi vào thực tế sau này.

Cuối cùng, em xin chúc Thầy/Cô luôn có thật nhiều sức khỏe, luôn tràn đầy nhiệt huyết và thành công hơn nữa trong quá trình giảng dạy.

TÓM TẮT

Trong thời đại công nghiệp 4.0, robot di động tự hành (Autonomous Mobile Robot) ngày càng được ứng dụng rộng rãi trong kho vận, dịch vụ và nghiên cứu. Nhằm củng cố kiến thức về hệ thống nhúng, điều khiển và lập trình ứng dụng,

nhóm thực hiện đã xây dựng một **xe robot tự hành nhỏ** sử dụng vi điều khiển **ESP32** làm trung tâm điều khiển, kết hợp với một **ứng dụng di động Flutter** làm giao diện điều hành.

Hệ thống cho phép người dùng chạm chọn điểm đích trên bản đồ; ứng dụng dùng thuật toán **A*** để tính đường đi ngắn nhất trên đồ thị waypoint, rồi gửi danh sách điểm trung gian xuống ESP32 qua giao thức **UDP** trên mạng WiFi do chính ESP32 phát ra. ESP32 bám theo lộ trình bằng cách kết hợp **odometry** từ encoder bánh xe và cảm biến góc **MPU-6050** (bộ lọc bù — complementary filter) để ước lượng vị trí và hướng theo thời gian thực, điều khiển hai động cơ DC qua mạch cầu **L298N**. Vị trí xe được phản hồi liên tục về ứng dụng (telemetry 10 Hz) để hiển thị trên bản đồ. Hệ thống còn tích hợp cơ chế **tự hiệu chuẩn** sai số động cơ/encoder lưu trên bộ nhớ NVS, và chế độ giả lập (simulator) để kiểm thử ứng dụng khi chưa có phần cứng.

Bên cạnh chế độ bám lộ trình theo bản đồ, hệ thống còn có **chế độ tự hành né vật cản (autonomous exploration)**: xe tự chạy thẳng, dùng cảm biến siêu âm **HC-SR04** gắn trên một **servo quét** để liên tục đo khoảng cách phía trước; khi gặp vật cản gần (dưới 30 cm) xe dừng lại, quay servo đo khoảng cách hai bên trái/phải rồi tự quyết định hướng đi thông thoáng hơn (rẽ 90° về bên rộng hơn, hoặc quay đầu 180° khi cả hai phía đều bị chặn, hoặc dừng hẳn khi bị kẹt). Người dùng bật/tắt chế độ này bằng một nút trên ứng dụng.

Thiết kế module hóa, chi phí thấp, không phụ thuộc Internet (mạng cục bộ), phù hợp làm nền tảng học thuật cho các bài toán định vị và dẫn đường robot trong quy mô phòng thí nghiệm.

MỤC LỤC

- LỜI CAM ĐOAN
- LỜI CẢM ƠN
- TÓM TẮT
- MỤC LỤC
- DANH MỤC HÌNH
- DANH MỤC BẢNG BIỂU
- DANH MỤC CÁC TỪ VIẾT TẮT

- **CHƯƠNG 1: TỔNG QUAN**
 - 1.1 Giới thiệu
 - 1.2 Tính cấp thiết của đề tài
 - 1.3 Mục tiêu nghiên cứu
 - 1.4 Nhiệm vụ nghiên cứu
 - 1.5 Đối tượng và phạm vi nghiên cứu
 - 1.6 Phương pháp nghiên cứu
 - 1.7 Bố cục đề tài
 - **CHƯƠNG 2: CƠ SỞ LÝ THUYẾT**
 - 2.1 Các giao thức kết nối sử dụng
 - 2.2 Các thuật toán và lý thuyết nền tảng
 - 2.3 Các phần mềm xây dựng và hỗ trợ
 - 2.4 Các thư viện sử dụng
 - **CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG**
 - 3.1 Yêu cầu và sơ đồ khối của hệ thống
 - 3.2 Thiết kế phần cứng
 - 3.3 Thiết kế phần mềm ứng dụng di động
 - 3.4 Giao thức truyền thông UDP
 - 3.5 Sơ đồ nối dây của hệ thống
 - 3.6 Lưu đồ giải thuật
 - **CHƯƠNG 4: THI CÔNG HỆ THỐNG**
 - 4.1 Danh sách linh kiện
 - 4.2 Thi công phần cứng và nối dây
 - 4.3 Lập trình firmware và ứng dụng
 - **CHƯƠNG 5: KẾT QUẢ - NHẬN XÉT - ĐÁNH GIÁ**
 - **CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN**
 - **TÀI LIỆU THAM KHẢO**
 - **PHỤ LỤC**
-

DANH MỤC HÌNH

Hình	Tên hình
Hình 2.1	Giao tiếp I2C giữa master và slave
Hình 2.2	Điều chế độ rộng xung PWM
Hình 2.3	Nguyên lý Trigger - Echo của HC-SR04
Hình 2.4	Mô hình xe vi sai (differential drive)
Hình 2.5	Minh họa thuật toán A* trên đồ thị
Hình 2.6	Giao diện PlatformIO trên VS Code
Hình 2.7	Ứng dụng Flutter
Hình 3.1	Sơ đồ khối hệ thống
Hình 3.2	Kiến trúc tổng thể firmware - app
Hình 3.3	Sơ đồ chân ESP32-WROOM-32
Hình 3.4	Kết nối ESP32 - L298N - động cơ
Hình 3.5	Kết nối encoder LM393
Hình 3.6	Kết nối MPU-6050 qua I2C
Hình 3.7	Kết nối cảm biến siêu âm HC-SR04
Hình 3.8	Bản đồ lưới waypoint 5×5 m
Hình 3.9	Bốn màn hình ứng dụng Flutter
Hình 3.10	Sơ đồ nguyên lý toàn hệ thống
Hình 3.11	Lưu đồ giải thuật chính (firmware)
Hình 3.12	Lưu đồ cập nhật odometry
Hình 3.13	Lưu đồ điều hướng node 3 tầng xoay
Hình 3.14	Lưu đồ chương trình con đi thẳng
Hình 3.15	Lưu đồ luồng điều hướng trên ứng dụng
Hình 3.16	Kết nối servo quét cảm biến với ESP32
Hình 3.17	Lưu đồ giải thuật chế độ tự hành né vật cản

Hình	Tên hình
Hình 4.1	Xe robot sau khi lắp ráp hoàn chỉnh
Hình 4.2	Màn hình bản đồ khi xe đang di chuyển

Ghi chú: các hình ảnh chụp thực tế, sơ đồ nguyên lý và ảnh giao diện được đánh dấu [CHÈN ẢNH ...] trong thân báo cáo — cần chèn ảnh chụp/ảnh xuất từ phần mềm khi đóng quyển.

DANH MỤC BẢNG BIỂU

Bảng	Tên bảng
Bảng 3.1	Phân bổ chân GPIO của ESP32
Bảng 3.2	Thông số kỹ thuật module L298N
Bảng 3.3	Bảng các giá trị nguồn của thiết bị
Bảng 3.4	Các tham số điều khiển chính của firmware
Bảng 3.5	Các giá trị trạng thái (status) telemetry
Bảng 3.6	Các tham số chế độ tự hành né vật cản
Bảng 4.1	Bảng liệt kê các linh kiện của hệ thống
Bảng 5.1	Kết quả đo độ chính xác quãng đường khi đi thẳng
Bảng 5.2	Kết quả đo độ chính xác xoay góc
Bảng 5.3	Kết quả đánh giá khả năng giữ hướng
Bảng 5.4	Kết quả điều hướng theo lộ trình node
Bảng 5.5	Kết quả đo khoảng cách bằng HC-SR04 theo ba hướng quét
Bảng 5.6	Kết quả thử nghiệm né vật cản tự hành
Bảng 5.7	Kết quả đo độ trôi góc yaw khi đứng yên
Bảng 5.8	Bảng mô tả trạng thái hoạt động của các khối

Danh mục công thức:

Công thức	Nội dung
Công thức 1	Chu vi và quãng đường mỗi xung encoder
Công thức 2	Khoảng cách từ cảm biến siêu âm
Công thức 3	Chuyển đổi gyro thô sang vận tốc góc
Công thức 4	Tích phân yaw từ con quay hồi chuyển
Công thức 5	Góc quay vi sai từ encoder
Công thức 6	Bộ lọc bù hợp nhất hướng
Công thức 7	Cập nhật tọa độ (x, y)
Công thức 8	Hàm chi phí và heuristic của A*
Công thức 9	Số xung mục tiêu cho một đoạn đường
Công thức 10	Hiệu chỉnh hướng (heading correction) khi đi thẳng
Công thức 11	Cập nhật hệ số bù động cơ (motor trim)
Công thức 12	Quy đổi góc servo sang độ rộng xung và duty LEDC

DANH MỤC CÁC TỪ VIẾT TẮT

Viết tắt	Tiếng Anh	Nghĩa tiếng Việt
ESP32	—	Vi điều khiển ESP32 (lõi kép, tích hợp WiFi/BLE)
GPIO	General Purpose Input/Output	Chân vào/ra tổng quát
I2C	Inter-Integrated Circuit	Giao tiếp giữa các mạch tích hợp
UART	Universal Asynchronous Receiver/Transmitter	Bộ thu/phát không đồng bộ
SPI	Serial Peripheral Interface	Giao tiếp ngoại vi nối tiếp
PWM	Pulse Width Modulation	Điều chế độ rộng xung
LEDC	LED Control (ESP32 PWM peripheral)	Bộ tạo PWM phần cứng của ESP32
PID	Proportional-Integral-Derivative	Bộ điều khiển tỉ lệ - tích phân - vi phân
IMU	Inertial Measurement Unit	Cảm biến quán tính (gia tốc + con quay)
MPU	Motion Processing Unit	Vi mạch cảm biến chuyển động (MPU-6050)
DLPF	Digital Low-Pass Filter	Bộ lọc thông thấp số (trong MPU-6050)
LSB	Least Significant Bit	Bit có trọng số thấp nhất (đơn vị giá trị thô)
dps	degrees per second	độ trên giây (đơn vị vận tốc góc)
UDP	User Datagram Protocol	Giao thức truyền không kết nối
JSON	JavaScript Object Notation	Định dạng trao đổi dữ liệu dạng văn bản
AP / SoftAP	(Software) Access Point	Điểm truy cập WiFi phần mềm
SSID	Service Set Identifier	Tên mạng WiFi

Viết tắt	Tiếng Anh	Nghĩa tiếng Việt
IP	Internet Protocol (address)	Địa chỉ giao thức Internet
NVS	Non-Volatile Storage	Bộ nhớ không mất dữ liệu (flash của ESP32)
ISR	Interrupt Service Routine	Chương trình phục vụ ngắt
A*	A-star algorithm	Thuật toán tìm đường A-sao
DC	Direct Current	Dòng điện một chiều (động cơ DC)
RPM	Revolutions Per Minute	Số vòng quay mỗi phút
ToF	Time of Flight	Cảm biến đo thời gian bay (laser)
PCB	Printed Circuit Board	Bảng mạch in
IDE	Integrated Development Environment	Môi trường phát triển tích hợp

CHƯƠNG 1: TỔNG QUAN

1.1 GIỚI THIỆU

Trong thời đại hiện nay, robot di động tự hành đang dần trở thành một thành phần quan trọng trong nhiều lĩnh vực: vận chuyển hàng hóa trong kho (AGV/AMR), robot dịch vụ, robot dò đường, cho tới các nền tảng nghiên cứu định vị và dẫn đường. Bài toán cốt lõi của một robot tự hành là: **biết mình đang ở đâu** (định vị), **biết phải đi tới đâu** (lập kế hoạch đường đi), và **đi tới đó một cách chính xác** (điều khiển bám lộ trình).

Các hệ thống robot công nghiệp thường sử dụng cảm biến đắt tiền (LiDAR, camera độ phân giải cao, GPS-RTK) và máy tính mạnh. Tuy nhiên, ở quy mô học thuật và sản phẩm chi phí thấp, hoàn toàn có thể xây dựng một robot tự hành cơ bản chỉ với vi điều khiển ESP32, encoder bánh xe, cảm biến quán tính giá rẻ và một điện thoại làm giao diện. Đề tài "**Xe robot tự hành định vị bằng odometry và tìm đường A***" ra đời nhằm nghiên cứu và triển khai một giải pháp robot tự

hành như vậy: dùng ESP32 làm bộ điều khiển trung tâm, kết hợp odometry (encoder + IMU) để định vị, thuật toán A* trên ứng dụng di động để tìm đường, và giao thức UDP để liên lạc thời gian thực giữa app và xe.

1.2 TÍNH CẤP THIẾT CỦA ĐỀ TÀI

Robot tự hành là một hướng công nghệ đang phát triển rất nhanh, gắn liền với tự động hóa và Industry 4.0. Tuy nhiên, để tiếp cận được lĩnh vực này, sinh viên thường gặp rào cản về chi phí phần cứng và độ phức tạp của thuật toán. Việc xây dựng một nền tảng robot tự hành **chi phí thấp, module hóa, dễ mở rộng** có ý nghĩa thực tiễn lớn: nó vừa là sản phẩm học tập giúp nắm vững các kiến thức nền (điều khiển động cơ, đọc cảm biến, định vị, lập kế hoạch đường đi, truyền thông không dây), vừa là khung sườn để phát triển tiếp các tính năng cao cấp hơn (tránh vật cản động, SLAM, đội hình nhiều robot).

Bên cạnh đó, việc dùng ESP32 — vi điều khiển tích hợp sẵn WiFi, giá rẻ, cộng đồng lớn — cùng Flutter cho ứng dụng đa nền tảng giúp hệ thống có khả năng mở rộng và tính linh hoạt cao trong tương lai. Đây chính là cơ sở để đề tài trở nên cấp thiết.

1.3 MỤC TIÊU NGHIÊN CỨU

Mục tiêu lớn: Thiết kế và xây dựng một xe robot tự hành sử dụng vi điều khiển ESP32, có khả năng tự định vị, tự tìm đường và bám theo lộ trình tới điểm đích do người dùng chọn trên ứng dụng di động.

Mục tiêu cụ thể:

- Nghiên cứu và lựa chọn phần cứng phù hợp: vi điều khiển, mạch lái động cơ, encoder, cảm biến quán tính, cảm biến khoảng cách.
- Xây dựng firmware ESP32 module hóa: điều khiển động cơ, đọc encoder/IMU/siêu âm, ước lượng odometry, máy trạng thái điều hướng giữa các node.
- Xây dựng ứng dụng di động (Flutter) hiển thị bản đồ, tính đường đi bằng A*, gửi lộ trình và hiển thị vị trí xe theo thời gian thực.
- Thiết kế giao thức truyền thông UDP độ trễ thấp giữa app và ESP32.
- Tích hợp cơ chế tự hiệu chuẩn để giảm sai số định vị, đảm bảo hệ thống hoạt động ổn định, chính xác trong phạm vi cho phép.

1.4 NHIỆM VỤ NGHIÊN CỨU

Trong quá trình thực hiện, đề tài tập trung vào các nhiệm vụ chính: khảo sát và phân tích các phương pháp định vị/dẫn đường cho robot di động; nghiên cứu các loại cảm biến phù hợp với ESP32; thiết kế sơ đồ khối và mạch nguyên lý của hệ thống; lập trình điều khiển động cơ và đọc cảm biến; cài đặt thuật toán odometry và bộ lọc bù; cài đặt thuật toán A* và bộ dựng waypoint trên ứng dụng; thiết kế giao thức UDP và cơ chế telemetry; kiểm thử hệ thống thực tế; đo đạc, tinh chỉnh tham số và đánh giá sai số; đề xuất hướng cải tiến.

1.5 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

Đối tượng nghiên cứu: hệ thống xe robot tự hành hai bánh vi sai (differential drive) sử dụng ESP32 làm trung tâm điều khiển, định vị bằng odometry encoder kết hợp IMU, dẫn đường bằng A* trên đồ thị waypoint.

Phạm vi nghiên cứu: giới hạn ở quy mô phòng thí nghiệm/trong nhà, trên mặt sàn phẳng, bản đồ biết trước dạng lưới điểm 5×5 m. Đề tài **không** sử dụng các công nghệ cao cấp như LiDAR, camera AI, GPS hay SLAM xây bản đồ động; định vị dựa hoàn toàn vào odometry tương đối (dead-reckoning). Việc thu gọn phạm vi như vậy giúp tập trung giải quyết trọn vẹn bài toán định vị - dẫn đường - bám lộ trình cơ bản với chi phí thấp.

1.6 PHƯƠNG PHÁP NGHIÊN CỨU

Đề tài sử dụng kết hợp các phương pháp: **phân tích - tổng hợp tài liệu** từ các nguồn học thuật và datasheet linh kiện; **mô phỏng - thực nghiệm** (chế độ simulator của ứng dụng cho phép kiểm thử phần mềm trước khi có phần cứng); **tính toán kỹ thuật điện tử** (lựa chọn linh kiện, tính toán quãng đường/góc quay, hiệu chỉnh sai số); và **lập trình nhúng** trên ESP32 (C++ với PlatformIO) song song với **lập trình ứng dụng** (Dart/Flutter). Hệ thống được phát triển theo từng giai đoạn (phase) và kiểm thử đơn vị (unit test) cho các module thuật toán quan trọng.

1.7 BỐ CỤC ĐỀ TÀI

Đề án được chia làm 6 chương như sau:

Chương 1: Tổng quan – Giới thiệu chung, nêu tính cấp thiết, xác định mục tiêu, đối tượng, phạm vi và phương pháp nghiên cứu.

Chương 2: Cơ sở lý thuyết – Trình bày các chuẩn giao tiếp (GPIO, I2C, PWM, Trigger-Echo, ngắt, WiFi/UDP), các thuật toán nền tảng (odometry vi sai, bộ lọc bù, A*), các phần mềm và thư viện sử dụng.

Chương 3: Thiết kế và xây dựng hệ thống – Từ yêu cầu, đưa ra sơ đồ khối, thiết kế phần cứng từng khối, kiến trúc phần mềm ứng dụng, giao thức UDP, sơ đồ nối dây và các lưu đồ giải thuật.

Chương 4: Thi công hệ thống – Liệt kê linh kiện, lắp ráp phần cứng, nối dây, lập trình firmware và ứng dụng.

Chương 5: Kết quả - Nhận xét - Đánh giá – Trình bày kết quả lý thuyết và thực hành, đánh giá hoạt động của từng khối.

Chương 6: Kết luận và hướng phát triển – Tổng kết ưu/nhược điểm và đề xuất hướng cải thiện, phát triển hệ thống.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 CÁC GIAO THỨC KẾT NỐI SỬ DỤNG

2.1.1 GPIO

GPIO (General Purpose Input/Output) là các chân tín hiệu được cấu hình bằng phần mềm để thực hiện chức năng ngõ vào (input) hoặc ngõ ra (output). Trong đề tài, GPIO của ESP32 được dùng cho hầu hết các kết nối số: điều khiển chiều quay động cơ (các chân IN1–IN4 của L298N), phát xung trigger cho cảm biến siêu âm, điều khiển còi buzzer, và đọc tín hiệu xung từ encoder.

ESP32 sử dụng mức logic **3.3 V**. Mỗi chân GPIO có thể cấu hình kéo lên (pull-up)/kéo xuống (pull-down) nội bộ, và quan trọng nhất với đề tài là khả năng tạo **ngắt ngoài** (interrupt) khi có cạnh tín hiệu — được dùng để đếm xung encoder. Cần lưu ý một số chân của ESP32 chỉ có chức năng input (GPIO34, GPIO35, GPIO36, GPIO39 — không có điện trở kéo nội bộ); trong đề tài chân **GPIO34** và **GPIO32** được chọn làm ngõ vào đọc encoder.

2.1.2 I2C

I2C (Inter-Integrated Circuit) là giao thức truyền thông nối tiếp đồng bộ, sử dụng hai đường:

- **SDA (Serial Data Line):** đường dữ liệu hai chiều.
- **SCL (Serial Clock Line):** đường xung nhịp do master cung cấp.

Hệ thống I2C gồm một master (ESP32) điều khiển bus và một hoặc nhiều slave được gán địa chỉ. Trong đề tài, I2C dùng để giao tiếp với cảm biến quán tính **MPU-6050** (địa chỉ mặc định **0x68**, hoặc **0x69** khi chân AD0 ở mức cao). ESP32 đặt chân **GPIO21 = SDA**, **GPIO22 = SCL** và chạy bus ở tốc độ 50 kHz để đảm bảo ổn định khi đi dây dài và có nhiễu từ động cơ.

[CHÈN HÌNH 2.1 — Sơ đồ giao tiếp I2C giữa master và các slave]

2.1.3 PWM và bộ LEDC của ESP32

PWM (Pulse Width Modulation – điều chế độ rộng xung) điều khiển công suất trung bình cấp cho tải bằng cách thay đổi tỉ lệ thời gian mức cao/thấp của tín hiệu (duty cycle). Tốc độ động cơ DC tỉ lệ với duty cycle của tín hiệu PWM cấp vào chân Enable của mạch cầu H.

ESP32 có bộ ngoại vi **LEDC** [8] chuyên tạo PWM phần cứng (không tốn CPU). Trong đề tài, hai kênh LEDC (kênh 0 cho bánh trái, kênh 1 cho bánh phải) được cấu hình **tần số 5 kHz, độ phân giải 8-bit** (duty 0–255), gắn vào hai chân Enable ENA/ENB của L298N. Giá trị PWM 0 là dừng, 255 là tốc độ tối đa.

Bộ LEDC còn được dùng để **điều khiển servo** quét cảm biến: một kênh LEDC riêng (kênh 2) được cấu hình **tần số 50 Hz, độ phân giải 16-bit** — đúng chuẩn tín hiệu servo RC (chu kỳ 20 ms, độ rộng xung 0,5–2,5 ms tương ứng góc 0–180°). Việc dùng kênh và bộ định thời (timer) tách biệt với hai kênh động cơ giúp servo và motor hoạt động đồng thời mà không xung đột tần số.

[CHÈN HÌNH 2.2 — Minh họa các mức duty cycle của tín hiệu PWM]

2.1.4 Trigger - Echo (cảm biến siêu âm)

Cảm biến siêu âm HC-SR04 đo khoảng cách theo nguyên lý phản hồi sóng âm với hai chân:

- **Trigger:** ngõ vào, vi điều khiển phát một xung mức cao **10 μs** để bắt đầu phép đo.
- **Echo:** ngõ ra, trả về một xung có độ rộng tỉ lệ với thời gian sóng âm đi tới vật cản và phản xạ về.

ESP32 đo độ rộng xung Echo (bằng hàm `pulseIn`) rồi suy ra khoảng cách. Vận tốc âm thanh trong không khí xấp xỉ 343 m/s (0,0343 cm/μs); vì sóng đi và về nên quãng đường tính được phải chia 2:

Công thức 2:

$$\text{Distance (cm)} = [t_echo(\mu s) \times 0,0343] / 2 = t_echo(\mu s) / 58,3$$

[CHÈN HÌNH 2.3 — Nguyên lý Trigger - Echo của HC-SR04]

2.1.5 Ngắt ngoài và đọc encoder

Encoder bánh xe tạo ra một chuỗi xung khi bánh quay. Để không bỏ sót xung khi vi điều khiển đang bận, đề tài dùng **ngắt ngoài (external interrupt)**: mỗi cạnh lên (RISING) trên chân encoder kích hoạt một ISR (Interrupt Service Routine) tăng bộ đếm xung. ISR được đặt trong RAM nội (`IRAM_ATTR`) để thực thi nhanh.

Do encoder quang LM393 một kênh (chỉ có ngõ D0) không cho biết chiều quay, firmware **lấy chiều quay từ lệnh điều khiển động cơ** (motor đang tiến thì xung cộng, lùi thì xung trừ). Để lọc nhiễu điện từ (EMI) do động cơ gây ra, ISR áp

dụng **chống dội phần mềm**: bỏ qua các xung cách nhau dưới **6000 μ s** (cho phép tối đa ~ 166 xung/s, dư biên rộng so với tốc độ chạy thực tế chỉ vài chục xung/s). Ngưỡng này còn lọc các **cạnh giả** do comparator LM393 (thiếu hysteresis) sinh ra ở mép khe đĩa.

2.1.6 WiFi và giao thức UDP

ESP32 tích hợp WiFi, có thể hoạt động ở chế độ **Access Point (SoftAP)** — tự phát ra một mạng WiFi để điện thoại kết nối trực tiếp, không cần router. Đề tài dùng chế độ này để tạo mạng cục bộ độ trễ thấp, hoạt động độc lập không cần Internet.

Trên mạng đó, dữ liệu được trao đổi bằng **UDP (User Datagram Protocol)** — giao thức không kết nối, không bắt tay, không đảm bảo thứ tự/độ tin cậy nhưng **độ trễ rất thấp**, rất phù hợp cho điều khiển robot thời gian thực: telemetry vị trí gửi 10 lần/giây có thể chấp nhận mất vài gói, còn lệnh quan trọng (lộ trình) được gửi lặp lại nhiều lần để bù.

2.2 CÁC THUẬT TOÁN VÀ LÝ THUYẾT NỀN TẢNG

2.2.1 Mô hình xe vi sai và odometry

Xe robot trong đề tài là loại **hai bánh vi sai (differential drive)**: hai bánh chủ động trái/phải điều khiển độc lập, hướng xe thay đổi nhờ chênh lệch tốc độ hai bánh. **Odometry** (đo hành trình) [7] là phương pháp ước lượng vị trí robot bằng cách tích lũy quãng đường mỗi bánh đo được từ encoder.

Gọi **dL**, **dR** là quãng đường bánh trái/phải đi được trong một chu kỳ, **L** là khoảng cách giữa hai bánh (wheel base). Khi đó:

- Quãng đường tịnh tiến trung tâm xe: $d = (dL + dR) / 2$
- Góc quay của xe: $\Delta\theta = (dR - dL) / L$

Cập nhật vị trí theo hệ trục (lưu ý hệ tọa độ của bản đồ có trục Y hướng xuống):

- $x += d \times \cos(\theta_mid)$
- $y -= d \times \sin(\theta_mid)$

với **θ_mid** là hướng trung bình trong chu kỳ. Quãng đường mỗi xung encoder quy đổi qua chu vi bánh xe (xem Công thức 1, mục 3.2.3).

[CHÈN HÌNH 2.4 — Mô hình động học xe vi sai]

2.2.2 Bộ lọc bù (Complementary Filter)

Odometry chỉ dựa trên encoder sẽ tích lũy sai số hướng rất nhanh (trượt bánh, đường kính bánh không đều). Ngược lại, con quay hồi chuyển (gyro) của IMU đo vận tốc góc rất tốt trong ngắn hạn nhưng bị **trôi (drift)** khi tích phân lâu dài. **Bộ lọc bù** kết hợp hai nguồn để bù khuyết điểm của nhau:

$$\theta_{\text{fused}} = \theta_{\text{enc}} + \alpha \times (\theta_{\text{imu}} - \theta_{\text{enc}})$$

Hệ số $\alpha = 0,35$ được chọn để **encoder làm chủ hướng (65%)** còn **IMU hiệu chỉnh (35%)**. Đây là lựa chọn thực nghiệm: encoder cho hướng ổn định khi đi thẳng, IMU kéo lại sai lệch khi bánh trượt. Trường hợp đặc biệt: khi **xoay tại chỗ** (hai bánh quay ngược chiều, encoder không đáng tin) thì dùng **IMU 100%**; khi **IMU mất kết nối** thì dùng **encoder 100%** làm phương án dự phòng.

2.2.3 Thuật toán tìm đường A*

A* (A-star) [6] là thuật toán tìm đường đi ngắn nhất trên đồ thị có trọng số, kết hợp chi phí thực tế đã đi $g(n)$ và một hàm ước lượng (heuristic) $h(n)$ tới đích:

Công thức 8:

$$f(n) = g(n) + h(n)$$

Trong đề tài, đồ thị là **lưới waypoint**, trọng số mỗi cạnh là khoảng cách Euclid giữa hai node; heuristic $h(n)$ cũng là khoảng cách Euclid từ node tới đích — đây là heuristic **chấp nhận được (admissible)** vì khoảng cách đường chim bay không bao giờ lớn hơn quãng đường thực, đảm bảo A* tìm ra đường tối ưu. Đề tài còn bổ sung một **chi phí phạt rẽ (turn penalty)**: khi hai đường có cùng độ dài, A* ưu tiên đường **ít phải rẽ hơn** để xe chạy mượt và giảm sai số tích lũy do mỗi lần xoay.

*[CHÈN HÌNH 2.5 — Minh họa quá trình mở rộng node của A trên đồ thị]**

2.3 CÁC PHẦN MỀM XÂY DỰNG VÀ HỖ TRỢ

2.3.1 PlatformIO trên Visual Studio Code

Firmware ESP32 được phát triển bằng **PlatformIO** — một hệ sinh thái phát triển nhúng chạy trên VS Code, hỗ trợ quản lý thư viện, biên dịch, nạp chương trình và giám sát Serial. So với Arduino IDE truyền thống, PlatformIO mạnh hơn ở quản

lý dự án nhiều file (module hóa), khai báo thư viện qua `platformio.ini`, và hỗ trợ nhiều môi trường build (đề tài có 2 môi trường: `esp32dev` cho firmware chính và `1298n_test` để test riêng mạch lái động cơ). Ngôn ngữ sử dụng là C++ trên framework Arduino-ESP32.

[CHÈN HÌNH 2.6 — Giao diện PlatformIO trên VS Code]

2.3.2 Flutter và ngôn ngữ Dart

Ứng dụng di động điều khiển được xây dựng bằng **Flutter** (ngôn ngữ **Dart**) của Google — framework đa nền tảng, từ một mã nguồn có thể build cho Android, iOS, Windows... Flutter cho phép vẽ giao diện bản đồ tùy biến (CustomPainter), xử lý chạm, quản lý trạng thái và giao tiếp UDP. Đề tài tận dụng Flutter để dựng 4 màn hình: Bản đồ, Kiểm tra kết nối, Điều khiển tay, và Telemetry/Hiệu chuẩn.

[CHÈN HÌNH 2.7 — Giao diện ứng dụng Flutter]

2.3.3 Proteus / Fritzing (thiết kế mạch)

Phần mềm thiết kế mạch (Proteus hoặc Fritzing) được dùng để vẽ sơ đồ nguyên lý kết nối ESP32 với các module và làm cơ sở cho việc đi dây/làm mạch. Do hệ thống chủ yếu lắp ghép từ các module có sẵn (L298N, MPU-6050, HC-SR04...), sơ đồ tập trung thể hiện kết nối chân giữa các khối.

2.4 CÁC THƯ VIỆN SỬ DỤNG

Firmware (C++ / Arduino-ESP32):

Thư viện	Vai trò trong đề tài
WiFi.h (built-in)	Tạo WiFi Access Point (SoftAP) cho ESP32
WiFiUdp.h (built-in)	Mở socket UDP gửi telemetry / nhận lệnh
Wire.h (built-in)	Giao tiếp I2C mức thấp với MPU-6050 (đọc/ghi thanh ghi trực tiếp, không dùng thư viện MPU bên ngoài)
Preferences.h (built-in)	Lưu dữ liệu hiệu chuẩn (motor trim, encoder scale) vào NVS flash
Hàm ledc*, attachInterrupt, pulseIn (core)	PWM phần cứng, ngắt encoder, đo xung siêu âm

Ứng dụng (Dart / Flutter):

Gói (package)	Phiên bản	Vai trò
udp	^2.0.0	Socket UDP gửi lệnh / nhận telemetry
path_provider	^2.1.0	Lấy đường dẫn file để ghi log CSV

Đáng chú ý: việc giao tiếp MPU-6050 được viết **thủ công ở mức thanh ghi** (đọc trực tiếp 14 byte từ thanh ghi 0x3B, tự đánh thức chip, đặt dải đo $\pm 250^\circ/\text{s}$, lọc DLPF ~ 44 Hz) thay vì dùng thư viện DMP, giúp hiểu sâu nguyên lý và chủ động xử lý lỗi đọc/mất kết nối.

CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

3.1 YÊU CẦU VÀ SƠ ĐỒ KHỐI CỦA HỆ THỐNG

3.1.1 Yêu cầu của hệ thống

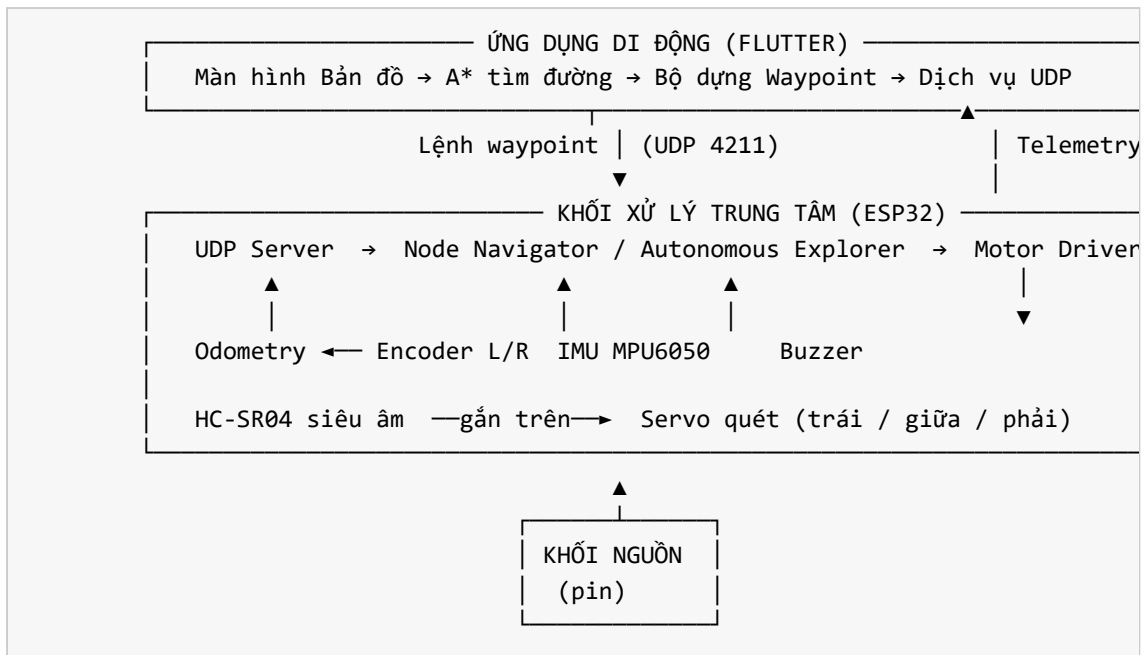
Hệ thống cung cấp các chức năng sau:

- Người dùng **chạm chọn điểm đích** trên bản đồ trong ứng dụng; ứng dụng tự **tính đường đi ngắn nhất** (A^*) và gửi lộ trình xuống xe.
- Xe **tự bám theo lộ trình**: tự xoay đúng hướng tại mỗi node và đi thẳng đúng quãng đường giữa các node, dựa trên odometry (encoder + IMU).
- **Hiển thị vị trí xe theo thời gian thực** trên bản đồ (telemetry 10 Hz), kèm trạng thái hoạt động và vệt đường đã đi.
- **Điều khiển tay** (tiến/lùi/rẽ/đi chéo) và **chế độ test từng bước** phục vụ kiểm tra.
- **Tự hiệu chuẩn** sai số cân bằng động cơ và tỉ lệ encoder, lưu lại qua các lần khởi động.
- **Chế độ tự hành né vật cản**: xe tự chạy thẳng và dùng siêu âm HC-SR04 trên servo quét để phát hiện, đo và **tự né vật cản** (rẽ 90° về bên thoáng hơn, quay đầu 180° hoặc dừng khi bị kẹt) — bật/tắt bằng nút trên ứng dụng. Đây là chế độ hoạt động độc lập với chế độ bám lộ trình A^* .
- Hoạt động được **không cần phần cứng** nhờ chế độ giả lập (simulator) trong ứng dụng.

3.1.2 Sơ đồ khối và chức năng mỗi khối

Sơ đồ khối tổng thể của hệ thống được trình bày ở Hình 3.1. Hệ thống gồm **hai phần lớn**: phần **firmware trên xe (ESP32)** và phần **ứng dụng trên điện thoại**, liên lạc qua WiFi/UDP.

Hình 3.1 — Sơ đồ khối hệ thống



3.1.3 Chức năng của từng khối

- **Khối xử lý trung tâm (ESP32):** "bộ não" của xe. Nhận lệnh từ app, đọc toàn bộ cảm biến, chạy odometry và máy trạng thái điều hướng, xuất tín hiệu điều khiển động cơ, gửi telemetry về app.
- **Khối động cơ (L298N + 2 DC motor):** nhận tín hiệu PWM + chiều quay từ ESP32 để dẫn động hai bánh chủ động.
- **Khối encoder (2 encoder quang):** đo số vòng/xung quay của từng bánh, cung cấp quãng đường cho odometry.
- **Khối cảm biến góc IMU (MPU-6050):** đo vận tốc góc để tính hướng (yaw) chính xác, hợp nhất với encoder qua bộ lọc bù.
- **Khối cảm biến khoảng cách (HC-SR04):** đo khoảng cách vật cản phía trước; trong chế độ tự hành, cảm biến được gắn trên servo để quét trái/phải tìm hướng thoát.
- **Khối servo quét cảm biến:** xoay cảm biến HC-SR04 sang trái/giữa/phải, giúp xe "nhìn" được khoảng cách hai bên mà không cần xoay cả thân xe.
- **Khối cảnh báo (buzzer):** phát tiếng "bíp" báo các sự kiện (tới node, tới đích, lỗi).
- **Khối nguồn (pin):** cấp nguồn cho động cơ và mạch điều khiển.
- **Ứng dụng di động:** giao diện người dùng — chọn đích, tính đường A*, gửi lộ trình, hiển thị vị trí xe và trạng thái theo thời gian thực; điều khiển tay,

hiệu chuẩn và bật/tắt chế độ tự hành.

3.1.4 Các hoạt động của hệ thống

Khi khởi động, ESP32 phát WiFi AP (SSID **RobotCar**, IP **192.168.4.1**), hiệu chuẩn IMU (đo bias gyro khi xe đứng yên) và đặt gốc tọa độ tại vị trí hiện tại. Người dùng mở ứng dụng, kết nối điện thoại vào mạng **RobotCar**, rồi chạm chọn node đích trên bản đồ. Ứng dụng tìm node gần điểm chạm nhất, chạy A* từ vị trí xe tới đích, dựng danh sách waypoint (tọa độ mét) và gửi xuống ESP32 qua UDP (gửi lặp để chống mất gói).

ESP32 nhận lộ trình và đưa vào **Node Navigator** — một máy trạng thái: tại mỗi node, xe tính hướng cần đi và **xoay** đúng hướng (cơ chế 3 tầng: bỏ qua nếu lệch nhỏ, xoay nhanh nếu lệch lớn, nhích tinh chỉnh nếu lệch vừa), rồi **đi thẳng** đúng quãng đường (đếm xung encoder) trong khi **giữ hướng bằng IMU**. Mỗi 20 ms, odometry cập nhật vị trí; mỗi 100 ms, ESP32 gửi telemetry (x, y, hướng, trạng thái) về app để vẽ vị trí xe. Khi tới node cuối, xe dừng và phát 3 tiếng bíp báo "đã tới đích" (**arrived**).

Ngoài luồng bấm lộ trình trên, hệ thống có thêm **chế độ tự hành né vật cản** do người dùng bật từ ứng dụng. Khi bật, ESP32 trao quyền điều khiển cho khối **Autonomous Explorer**: xe tự chạy thẳng và đo khoảng cách phía trước; gặp vật cản gần thì dừng, dùng servo quét đo hai bên rồi tự chọn hướng đi mới (chi tiết thuật toán ở mục 3.6.6). Hai chế độ (bấm lộ trình A* và tự hành né vật cản) loại trừ lẫn nhau — khi nhận lệnh chế độ này thì chế độ kia tự dừng.

3.2 THIẾT KẾ PHẦN CỨNG

Bảng 3.1 tổng hợp toàn bộ phân bổ chân GPIO của ESP32 trong hệ thống (theo `firmware/src/config.h`).

Bảng 3.1 — Phân bổ chân GPIO của ESP32

Khối	Tín hiệu	Chân ESP32
Động cơ trái	IN1 / IN2 / ENA (PWM)	GPIO12 / GPIO13 / GPIO14
Động cơ phải	IN3 / IN4 / ENB (PWM)	GPIO26 / GPIO27 / GPIO25
Encoder	Trái / Phải (D0)	GPIO34 / GPIO32
IMU MPU-6050	SDA / SCL (I2C)	GPIO21 / GPIO22
Siêu âm HC-SR04	Trig / Echo	GPIO5 / GPIO18
Servo quét cảm biến	Tín hiệu PWM (LEDC kênh 2)	GPIO19
Cảnh báo	Buzzer	GPIO4

3.2.1 Khối xử lý trung tâm (ESP32)

Chức năng: trung tâm điều khiển, chạy toàn bộ firmware ở chu kỳ vòng lặp **50 Hz (20 ms)**.

Lựa chọn linh kiện: đề tài sử dụng board **ESP32 DevKit (ESP32-WROOM-32, 38 chân)** [1], [2] vì các lý do:

- **Hiệu năng cao:** lõi kép Xtensa 240 MHz, đủ sức chạy đồng thời odometry, máy trạng thái điều hướng và UDP.
- **Tích hợp WiFi:** phát được Access Point để app kết nối trực tiếp, không cần module mạng ngoài — đây là yếu tố then chốt giúp loại bỏ router.
- **Nhiều GPIO và ngoại vi:** đủ chân cho 2 động cơ (6 chân), 2 encoder, I2C, siêu âm, buzzer; có bộ LEDC tạo PWM phần cứng và nhiều chân hỗ trợ ngắt.
- **Hỗ trợ NVS flash:** lưu dữ liệu hiệu chuẩn không mất khi tắt nguồn.
- **Phổ biến, chi phí thấp, cộng đồng lớn,** dễ lập trình bằng C++ trên PlatformIO.

[CHÈN HÌNH 3.3 — Sơ đồ chân ESP32-WROOM-32]

3.2.2 Khối động cơ (mạch lái L298N + 2 động cơ DC)

Chức năng: chuyển tín hiệu điều khiển logic của ESP32 thành dòng đủ lớn để quay hai động cơ DC.

Lựa chọn linh kiện: dùng module **L298N** (mạch cầu H đôi) [4]. L298N điều khiển độc lập hai động cơ, mỗi động cơ có 2 chân chiều quay (IN) và 1 chân Enable nhận PWM. So với DRV8833/TB6612, L298N có sẵn, rẻ, chịu dòng tốt, tản nhiệt tích hợp, phù hợp động cơ DC giảm tốc cỡ nhỏ.

Bảng 3.2 — Thông số kỹ thuật module L298N

Thuộc tính	Giá trị
Điện áp động lực	5 – 35 V
Dòng đỉnh mỗi kênh	2 A
Số kênh (động cơ)	2 (cầu H đôi)
Tín hiệu điều khiển	Logic 3.3 V / 5 V tương thích
Có ổn áp 5V on-board	Có (cấp ngược lại cho ESP32)

Nguyên lý điều khiển (theo `motor_driver.cpp`): mỗi bánh có một giá trị tốc độ trong khoảng **-255...255**. Dấu quyết định chiều (đặt IN1/IN2 hoặc IN3/IN4), độ lớn là duty PWM cấp vào chân Enable qua kênh LEDC (5 kHz, 8-bit). Firmware còn cung cấp hàm **tăng/giảm tốc từ từ (slew rate)** — mỗi vòng lặp chỉ thay đổi PWM tối đa 18 đơn vị — để tránh giật khi khởi động và giảm trượt bánh.

Mô tả kết nối:

- IN1 = GPIO12, IN2 = GPIO13, ENA = GPIO14 (động cơ trái).
- IN3 = GPIO26, IN4 = GPIO27, ENB = GPIO25 (động cơ phải).
- Chân động lực 12V/VS của L298N lấy từ pin; 5V out của L298N có thể cấp ngược cho ESP32. GND của L298N, ESP32 và pin **phải nối chung (common ground)**.

[CHÈN HÌNH 3.4 — Sơ đồ kết nối ESP32 - L298N - hai động cơ DC]

3.2.3 Khối encoder

Chức năng: đo quãng đường mỗi bánh, là nguồn dữ liệu chính cho odometry.

Lựa chọn linh kiện: dùng **encoder quang LM393** với đĩa mã hóa **20 khe** (20 xung/vòng), gắn trên trục mỗi bánh. Đây là loại encoder số một kênh (chỉ có ngõ

D0), rẻ, dễ lắp; chiều quay được suy ra từ lệnh động cơ nên không cần kênh thứ hai.

Tính toán quãng đường: với đường kính bánh **6,5 cm**:

Công thức 1:

$$\text{Chu vi} = \pi \times D = \pi \times 6,5 = 20,42 \text{ cm}$$

$$\text{cm mỗi xung} = \text{Chu vi} / (\text{xung mỗi vòng}) = 20,42 / 20 = 1,021 \text{ cm}$$

Khoảng cách giữa hai node là **0,5 m**, tương đương $0,5 \text{ m} / 1,021 \text{ cm} \approx 49 \text{ xung}$. Firmware tự suy `PULSES_PER_NODE` từ `CM_PER_PULSE` (một nguồn sự thật duy nhất): khi thay đường kính bánh hay số xung mỗi vòng thì số xung mục tiêu tự cập nhật theo, tránh sai lệch khi chỉnh tay. Giá trị được kiểm chứng bằng đo đạc thực tế (chạy quãng 0,5 m, đối chiếu thước) và tinh chỉnh qua hằng số `ENCODER_PULSES_PER_REV`.

Xử lý nhiễu: ISR đếm xung trên cạnh lên, áp dụng chống dội **6000 μs** để loại nhiễu EMI từ động cơ và cạnh giả do comparator LM393 (thiếu hysteresis) sinh ra ở mép khe. Việc đọc bộ đếm được bảo vệ bằng tắt/bật ngắt tạm thời (atomic) để tránh tranh chấp với ISR.

[CHÈN HÌNH 3.5 — Sơ đồ kết nối encoder LM393 với ESP32]

3.2.4 Khối cảm biến góc IMU (MPU-6050)

Chức năng: đo hướng (yaw) của xe để hiệu chỉnh odometry và giữ hướng khi đi thẳng / xoay đúng góc.

Lựa chọn linh kiện: dùng **MPU-6050** [3] (6 trục: 3 gia tốc + 3 con quay hồi chuyển), giao tiếp I2C, giá rẻ, rất phổ biến. Đề tài chỉ dùng **trục gyro Z** để tính yaw.

Nguyên lý (theo `imu_mpu6050.cpp`):

- Khởi động: đánh thức chip, đặt tần số lấy mẫu **100 Hz**, bộ lọc DLPF $\sim 44 \text{ Hz}$, dải đo con quay $\pm 250^\circ/\text{s}$ (độ nhạy 131 LSB/($^\circ/\text{s}$)).
- **Hiệu chuẩn bias:** khi xe đứng yên, đọc ~ 1000 mẫu gyro Z để tính giá trị lệch 0 (bias), loại bỏ trôi tĩnh.
- **Tính yaw:** mỗi chu kỳ, đọc giá trị gyro thô, trừ bias, đổi sang rad/s rồi tích phân theo thời gian.

Công thức 3 (đổi gyro thô → vận tốc góc):

$$\omega_z = (gz_raw - bias_z) \times 1 / (131 \times 180/\pi) \text{ [rad/s]}$$

Công thức 4 (tích phân yaw):

$$\theta_yaw += \omega_z \times \Delta t$$

Để chống trôi khi đứng yên, áp dụng **deadband 0,01 rad/s** (bỏ qua vận tốc góc rất nhỏ). Firmware còn theo dõi sức khỏe cảm biến: nếu đọc lỗi liên tiếp 5 lần thì đánh dấu "mất kết nối" và odometry tự chuyển sang dùng encoder 100%.

Mô tả kết nối: VCC → 3.3V/5V; GND → GND chung; SDA → GPIO21; SCL → GPIO22.

[CHÈN HÌNH 3.6 — Sơ đồ kết nối MPU-6050 qua I2C]

3.2.5 Khối cảm biến khoảng cách (HC-SR04)

Chức năng: đo khoảng cách vật cản phía trước để phục vụ **chế độ tự hành né vật cản** — phát hiện chướng ngại, dừng kịp thời và đo khoảng cách hai bên (qua servo quét) để chọn hướng đi.

Lựa chọn linh kiện: dùng **HC-SR04** [5] (siêu âm), giao tiếp số đơn giản (Trig/Echo), tầm đo 2–400 cm, rẻ, dễ tích hợp, không nhạy với ánh sáng môi trường (khác cảm biến hồng ngoại). So với cảm biến ToF laser (VL53L0X) đắt hơn, HC-SR04 đủ dùng cho mục tiêu phát hiện vật cản cơ bản.

Nguyên lý (theo `ultrasonic_hcsr04.cpp`): phát xung trigger 10 μ s, đo độ rộng xung Echo bằng `pulseIn` (timeout 30 ms $\approx$ ngoài tầm), quy đổi sang cm theo Công thức 2 (vận tốc âm $\sim 0,0343$ cm/ μ s, chia 2 vì sóng đi và về). Firmware cung cấp hai cách đọc tùy ngữ cảnh:

- **Đọc kỹ (`readDistanceCm`)** — lấy **trung vị của 5 mẫu** (median filter, có nghỉ giữa các mẫu) để khử nhiễu; dùng khi xe **đứng yên** quét hai bên, nơi cần độ chính xác cao.
- **Đọc nhanh (`readQuickCm`)** — chỉ một lần đo, không nghỉ, để **không làm nghẽn vòng lặp 50 Hz** khi xe đang chạy thẳng và cần đo liên tục. Khi phát hiện vật cản, firmware đọc thêm một lần xác nhận để loại nhiễu một lần trước khi quyết định dừng.

Ngưỡng quyết định trong chế độ tự hành: vật cản phía trước **< 30 cm** thì dừng và quét; cần **≥ 25 cm** ở cả hai bên mới đủ chỗ quay đầu 180° (chi tiết ở mục 3.6.6).

Ngoài ra mã nguồn vẫn giữ tùy chọn **dừng khẩn cấp** độc lập (`USE_ULTRASONIC`, ngưỡng 10 cm / cảnh báo 20 cm) dành cho chế độ bám lộ trình, mặc định tắt để không làm chậm vòng lặp khi không cần.

[CHÈN HÌNH 3.7 — Sơ đồ kết nối HC-SR04 với ESP32]

3.2.6 Khối servo quét cảm biến

Chức năng: xoay cảm biến HC-SR04 sang **trái / giữa / phải** để xe "nhìn" được khoảng cách ba hướng mà **không cần xoay cả thân xe** — giúp khâu quyết định né vật cản nhanh và ít hao pin hơn.

Lựa chọn linh kiện: dùng servo **SG90** (xoay 180°), điều khiển bằng một dây tín hiệu PWM 50 Hz, giá rẻ và rất phổ biến. Thông số chính: điện áp **4,8–6 V**, mô-men **~1,2–2,4 kg·cm**, đủ để xoay nhẹ cụm cảm biến HC-SR04; trọng lượng nhỏ (~9 g) phù hợp gắn ở mũi xe.

Bảng 3.6 — Thông số servo SG90 và tham số chế độ tự hành

Thông số	Giá trị
Loại servo / góc quay	SG90 / 0–180°
Điện áp hoạt động	4,8 – 6 V
Mô-men	~1,2 – 2,4 kg·cm
Tín hiệu điều khiển	PWM 50 Hz (xung 0,5–2,5 ms), LEDC kênh 2, 16-bit
Góc giữa / trái / phải	90° / 150° / 30°
Thời gian chờ servo ổn định	~350 ms trước mỗi lần đo
Ngưỡng dừng phía trước	< 30 cm
Khoảng trống tối thiểu để quay đầu	≥ 25 cm hai bên
Tốc độ chạy thẳng tự hành	PWM 160
Góc né khi rẽ	90° (trái/phải) hoặc 180° (quay đầu)

Nguyên lý điều khiển (theo `sensor_pan_servo.cpp`): thay vì dùng thư viện servo ngoài, đề tài điều khiển trực tiếp bằng **LEDC thô** trên **kênh 2 riêng** (50

Hz, độ phân giải 16-bit) để đồng bộ phong cách với khối động cơ và tránh xung đột bộ định thời. Góc mong muốn (0–180°) được quy đổi sang độ rộng xung rồi sang duty LEDC:

Công thức 12 (góc servo → xung → duty):

$$\text{pulse}(\mu\text{s}) = 500 + (2000 \times \text{góc}) / 180$$

$$\text{duty} = \text{pulse} \times (2^{16} - 1) / 20000$$

Ba vị trí dừng trong chế độ tự hành: **giữa** = 90° (nhìn thẳng khi chạy), **trái** = 150°, **phải** = 30°. Sau mỗi lần ra lệnh xoay, firmware **chờ ~350 ms cho servo ổn định** rồi mới đo siêu âm để tránh đọc sai lúc cánh tay còn đang quay. *(Các góc trái/phải có thể tinh chỉnh theo cách lắp cơ khí thực tế.)*

[CHÈN HÌNH 3.16 — Sơ đồ kết nối servo quét cảm biến với ESP32]

3.2.7 Khối cảnh báo (buzzer)

Chức năng: phản hồi âm thanh cho người dùng. Buzzer (chân **GPIO4**) phát các chuỗi bíp ngắn báo sự kiện: **1 bíp** khi qua một node, **3 bíp** khi tới đích (arrived). Cơ chế bíp được lập lịch không chặn (non-blocking): bật 75 ms, nghỉ 95 ms giữa các tiếng, không làm gián đoạn vòng lặp điều khiển.

3.2.8 Khối nguồn

Hệ thống dùng nguồn pin cấp cho cả động lực và điều khiển. Bảng 3.3 liệt kê mức điện áp các thiết bị.

Bảng 3.3 — Các giá trị nguồn của thiết bị

Thiết bị	Điện áp hoạt động
ESP32	3.3 V (logic) / 5 V (cấp qua VIN)
L298N (động lực)	5 – 35 V (đề tài dùng ~7.4 V pin)
Động cơ DC giảm tốc	3 – 6 V
MPU-6050	3.3 – 5 V
HC-SR04	5 V
Servo SG90/MG90S	5 V (nên cấp riêng/tụ lọc để tránh sụt áp)
Encoder LM393	3.3 – 5 V
Buzzer	3.3 – 5 V

Phương án cấp nguồn: pin (ví dụ 2 cell Li-ion 18650, ~7.4 V) cấp vào chân động lực L298N; ngõ ổn áp 5V của L298N cấp ngược cho ESP32 (qua VIN). Toàn bộ GND nối chung. ESP32 tự có các bộ ổn áp tạo mức 3.3 V cho logic và cảm biến. Lưu ý phần mềm có tắt cảnh báo sụt áp (brown-out) của ESP32 để tránh reset khi động cơ khởi động kéo dòng lớn.

3.3 THIẾT KẾ PHẦN MỀM ỨNG DỤNG DI ĐỘNG (FLUTTER)

Ứng dụng được xây dựng bằng **Flutter** [9], tổ chức theo mô hình phân lớp rõ ràng (models – planning – services – screens), giúp dễ kiểm thử và bảo trì.

Các lớp chính:

- **models:** `RobotState` (pose x, y, θ ; khoảng cách vật cản; trạng thái — kể cả các trạng thái tự hành `exploring/scanning/avoiding/stuck`), `RoadMap` (Node, Edge, các cạnh bị chặn lúc chạy).
- **planning:** `AStar` (tìm đường), `RoadMap loader` (đọc bản đồ JSON), `WaypointBuilder` (đổi danh sách node thành danh sách tọa độ mét).
- **services:** `UdpService` (gửi lệnh / nhận telemetry qua UDP, có hàm `sendAutonomousCommand` bật/tắt chế độ tự hành) và `SimulatorService` (giả lập xe chạy 0,5 m/s khi không có phần cứng) — cùng hiện thực giao diện `ITransportService` nên có thể hoán đổi.
- **screens:** 4 màn hình điều hành.

Bốn màn hình ứng dụng (Hình 3.9):

1. **Bản đồ (Map):** vẽ đồ thị waypoint, vị trí xe thời gian thực + vệt đường đã đi; chạm để chọn đích $\rightarrow A^* \rightarrow$ gửi lộ trình.
2. **Kiểm tra kết nối (Connection Test):** kiểm tra luồng telemetry/UDP với ESP32.
3. **Điều khiển tay (Manual Control):** nút tiến/lùi/rẽ/đi chéo/dừng.
4. **Tự hành (Autonomous):** nút Start/Stop bật/tắt chế độ tự hành né vật cản, kèm bảng telemetry trực tiếp (trạng thái, khoảng cách vật cản, hướng, tọa độ, tình trạng IMU). Màn hình này thay cho màn hình test bánh xe (Wheels) ở phiên bản trước — vốn chỉ dùng để chạy thử từng bánh và đã trở nên dư thừa.

Chế độ giả lập: thông qua biến `USE_SIMULATOR`, ứng dụng có thể chạy hoàn toàn không cần ESP32 (xe ảo cập nhật 10 Hz) — rất hữu ích để phát triển và demo giao diện.

[CHÈN HÌNH 3.8 — Bản đồ lưới waypoint 5×5 m] [CHÈN HÌNH 3.9 — Bốn màn hình của ứng dụng Flutter]

Bản đồ demo: lưới điểm waypoint 5×5 m, bước 0,5 m, gồm 105 node với các hành lang giữa được chọn lọc; các cạnh đều **hai chiều (bidirectional)** nối các node kề nhau theo phương ngang/dọc. Bản đồ được lưu dạng JSON (`environment.json`), đóng gói sẵn vào ứng dụng và đồng bộ với firmware.

3.4 GIAO THỨC TRUYỀN THÔNG UDP

ESP32 phát WiFi AP: SSID `RobotCar`, mật khẩu `12345678`, IP cố định `192.168.4.1`. Hai chiều dữ liệu dùng hai cổng UDP riêng:

Hướng	Cổng	Tần suất	Nội dung
ESP32 $\rightarrow$ App (telemetry)	4210	10 Hz	Vị trí, hướng, trạng thái, khoảng cách vật cản
App $\rightarrow$ ESP32 (lệnh)	4211	Khi cần	Lộ trình, điều khiển tay, tự hành, hiệu chuẩn, reset

Gói telemetry (JSON, ESP32 $\rightarrow$ App):

```
{ "pose": { "x": 0.3500, "y": 0.1200, "theta": 1.5700 },  
  "heading_deg": 90.0, "node_index": 2, "route_size": 5,  
  "obstacle_dist_cm": 200.0, "status": "moving",  
  "imu_connected": true, "calibrated": true }
```

(Trường `obstacle_dist_cm` báo khoảng cách phía trước đo được khi đang ở chế độ tự hành; ngoài chế độ này (bám lộ trình thường) nó báo giá trị tối đa `200.0` cm vì siêu âm không được đọc liên tục.)

Gói lệnh lộ trình (JSON, App → ESP32):

```
{ "seq": 1, "waypoints": [ { "x": 0.5, "y": 0.0 }, { "x": 1.0, "y": 0.0 } ] }
```

Gói lệnh bật/tắt tự hành (JSON, App → ESP32):

```
{ "seq": 42, "type": "autonomous", "command": "start" }
```

Trường `command` nhận `"start"` (bật chế độ tự hành né vật cản) hoặc `"stop"` (tắt, dừng xe và đưa servo về giữa).

Cơ chế đảm bảo tin cậy trên nền UDP:

- **Chống trùng lặp (dedup):** mỗi lệnh có số thứ tự `seq`; ESP32 ghi nhớ `seq` cuối, bỏ qua nếu trùng. App gửi lặp một lệnh nhiều lần để chống mất gói mà không gây thực thi nhiều lần.
- **Chia mảnh lộ trình (route_chunk):** lộ trình dài được chia thành nhiều gói nhỏ (kèm `chunk_index`, `chunk_count`, `start`, `total`); ESP32 ghép lại đủ mới thực thi — tránh tràn kích thước gói UDP.
- Các loại lệnh khác: `manual` (điều khiển tay, kèm tốc độ và lệnh hướng), `autonomous` (bật/tắt tự hành né vật cản), `step` (test một bước), `reset_pose` (đặt lại gốc tọa độ), `calibrate` (hiệu chuẩn IMU/động cơ/encoder), `ping`. Khi nhận `autonomous start`, các chế độ khác (tay/lộ trình) tự dừng; ngược lại bất kỳ lệnh tay/lộ trình/reset nào cũng tự tắt chế độ tự hành.

Bảng 3.5 — Các giá trị trạng thái (status) trong telemetry

Trạng thái	Ý nghĩa
idle	Đứng yên, sẵn sàng
moving	Đang di chuyển (xoay hoặc đi thẳng)
arrived	Đã tới đích
exploring	Tự hành: đang chạy thẳng dò đường
scanning	Tự hành: đang quét servo đo hai bên
avoiding	Tự hành: đang xoay né vật cản (90° hoặc 180°)
stuck	Tự hành: bị kẹt (cả ba phía đều hẹp), đã dừng
emergency_stop	Dừng khẩn cấp do vật cản
imu_missing	Mất kết nối IMU (đang chạy dự phòng encoder)
calibrating / calibrated	Đang / đã hiệu chuẩn
rotation_timeout	Lỗi: xoay quá lâu (motor yếu/kẹt)

3.5 SƠ ĐỒ NỐI DÂY CỦA HỆ THỐNG

Sơ đồ nguyên lý toàn hệ thống thể hiện ESP32 ở trung tâm, kết nối tới L298N (6 dây điều khiển), hai encoder (2 dây tín hiệu), MPU-6050 (2 dây I2C), HC-SR04 (2 dây Trig/Echo), servo quét cảm biến (1 dây tín hiệu GPIO19), buzzer (1 dây) và khối nguồn. Toàn bộ tuân theo phân bổ chân ở Bảng 3.1. HC-SR04 được gắn lên cánh tay servo để quay theo servo.

[CHÈN HÌNH 3.10 — Sơ đồ nguyên lý toàn hệ thống (vẽ trên Proteus/Fritzing)]

Bảng 3.4 — Các tham số điều khiển chính của firmware (theo `config.h`)

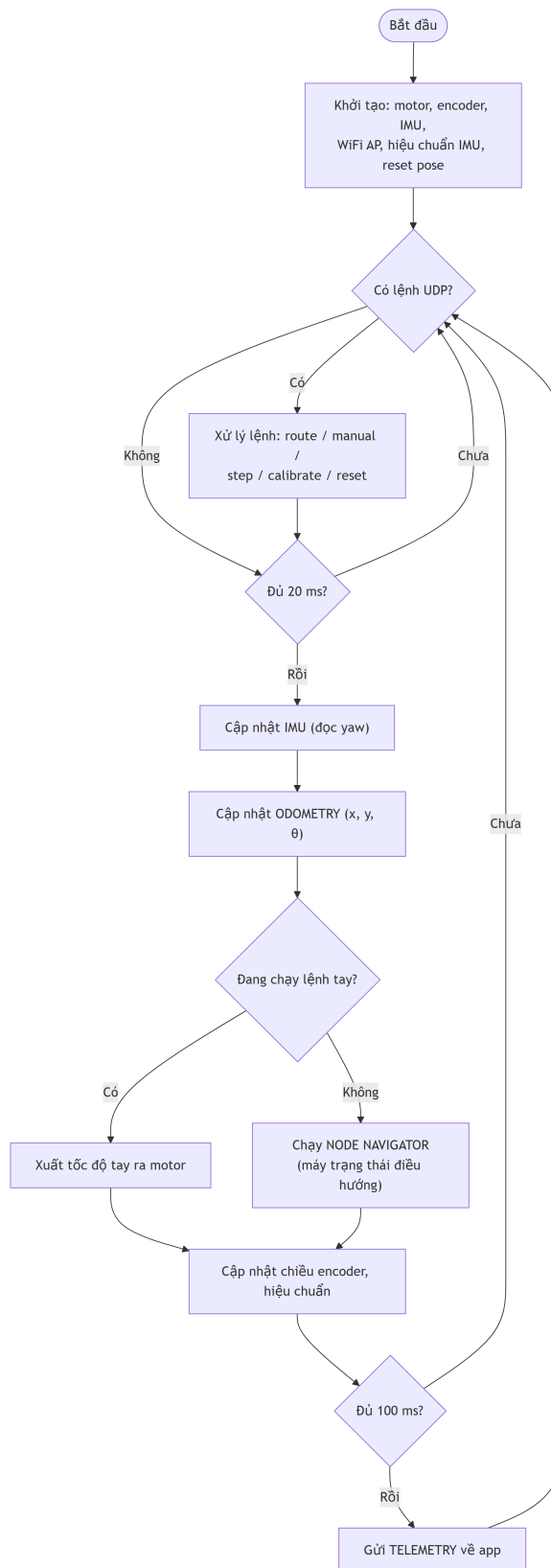
Tham số	Giá trị	Ý nghĩa
Chu kỳ vòng lặp	20 ms (50 Hz)	Tần suất cập nhật điều khiển/odometry
Chu kỳ telemetry	100 ms (10 Hz)	Tần suất gửi vị trí về app
Wheel base	10,5 cm	Khoảng cách 2 bánh (tính góc quay)
cm/xung	1,021 cm	Quãng đường mỗi xung encoder
Xung/node	52	Số xung đi hết 0,5 m
ALPHA (lọc bù)	0,35	Tỉ trọng IMU khi hợp nhất hướng
Tốc độ đi thẳng	PWM 175 (135–220)	Tốc độ cơ sở, giảm dần khi gần đích
Ngưỡng bỏ qua xoay	10°	Lệch nhỏ hơn → đi thẳng luôn
Ngưỡng xoay nhanh	25°	Lớn hơn → xoay nhanh trước
Dung sai xoay xong	4°	Sai số hướng chấp nhận được

3.6 LƯU ĐỒ GIẢI THUẬT

3.6.1 Lưu đồ giải thuật chính (firmware)

Sau khi khai báo thư viện và khởi tạo (motor, encoder, IMU, WiFi AP, hiệu chuẩn IMU, đặt gốc tọa độ), chương trình vào vòng lặp chính 50 Hz: nhận lệnh UDP → cập nhật IMU → cập nhật odometry → chạy máy trạng thái điều hướng (hoặc thực thi lệnh tay) → gửi telemetry.

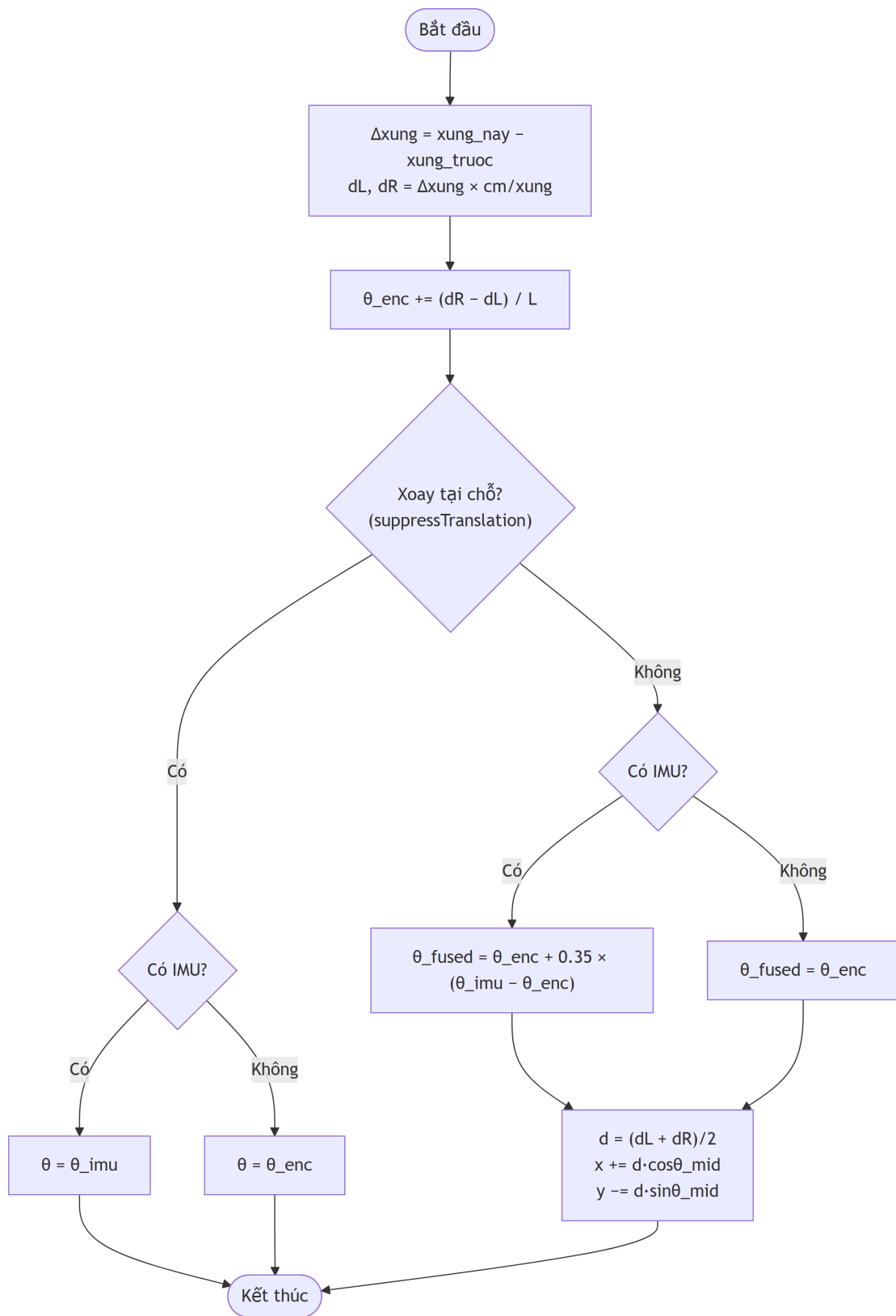
Hình 3.11 — Lưu đồ giải thuật chính



3.6.2 Lưu đồ chương trình con cập nhật Odometry

Mỗi chu kỳ, tính độ thay đổi xung trái/phải $\rightarrow$ quãng đường $d_L, d_R \rightarrow$ góc quay encoder. Nếu đang xoay tại chỗ thì chỉ cập nhật hướng (ưu tiên IMU); nếu đi thẳng/cong thì hợp nhất hướng bằng bộ lọc bù rồi cập nhật tọa độ (x, y).

Hình 3.12 — Lưu đồ cập nhật odometry



Các công thức odometry cốt lõi:

Công thức 5 (góc quay vì sai): $\Delta\theta_{enc} = (dR - dL) / L$

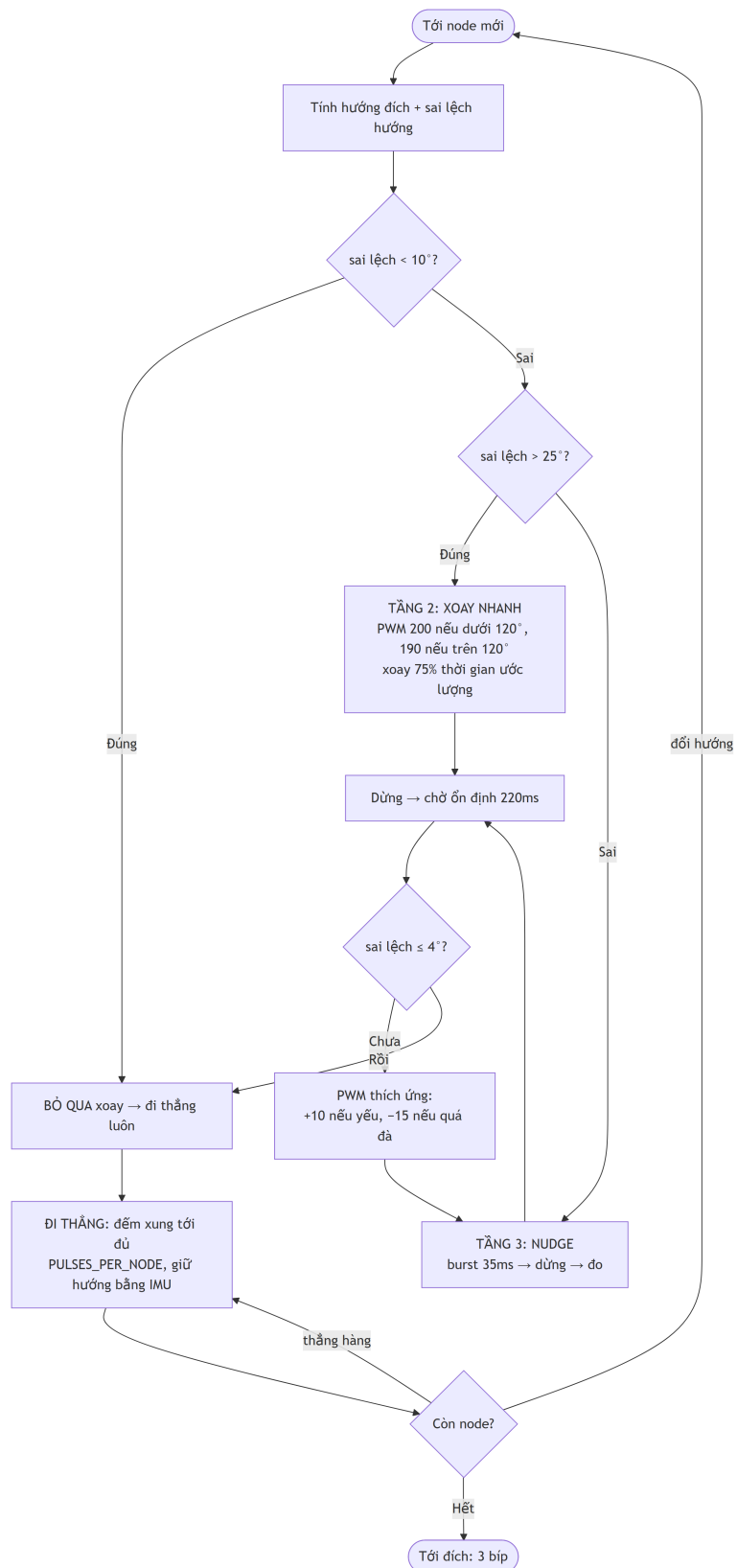
Công thức 6 (lọc bù): $\theta_{fused} = \theta_{enc} + 0,35 \times (\theta_{imu} - \theta_{enc})$

Công thức 7 (cập nhật vị trí): $x += d \times \cos(\theta_{mid}); y -= d \times \sin(\theta_{mid})$

3.6.3 Lưu đồ điều hướng node — cơ chế 3 tầng xoay

Đây là phần "trí tuệ" của firmware. Tại mỗi node, navigator tính **hướng cần đi** và **sai lệch hướng**, rồi chọn một trong ba chiến lược xoay tùy độ lớn sai lệch, sau đó **đi thẳng** tới node tiếp theo.

Hình 3.13 — Lưu đồ điều hướng node 3 tầng xoay



Giải thích ba tầng:

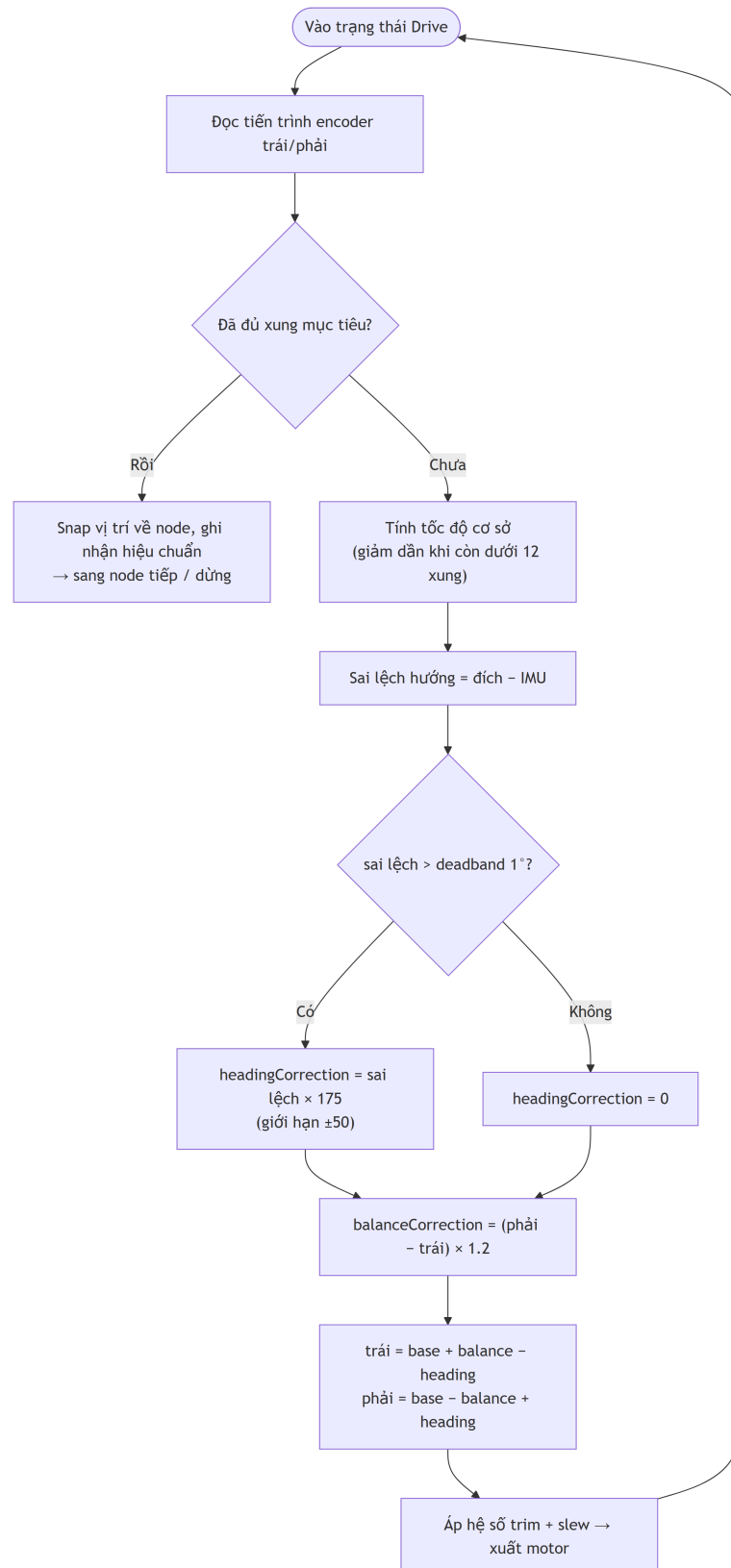
- **Tầng 1 – Bỏ qua ($< 10^\circ$):** sai lệch nhỏ thì không xoay, đi thẳng luôn và để khâu hiệu chỉnh hướng tự kéo lại. Giúp xe chạy mượt, không khựng tại mỗi node.
- **Tầng 2 – Xoay nhanh ($> 25^\circ$):** xoay liên tục ở PWM cao theo **thời gian ước lượng** ($150^\circ/\text{s} \times 75\%$ để tránh trượt quá), kèm phát hiện **vượt đích (cross-target)** để dừng ngay khi sai lệch đổi dấu — xử lý đúng trường hợp quay đầu 180° (chống lỗi "angle wrapping").
- **Tầng 3 – Nudge tinh chỉnh (10° – 25° hoặc phần dư sau xoay nhanh):** lặp chu trình **cấp xung 35 ms** → **dừng** → **chờ 220 ms cho IMU ổn định** → **đo hướng**. PWM **tự thích ứng**: tăng +10 khi xoay không đủ (thẳng mà sát tĩnh), giảm -15 khi xoay quá đà. Robot luôn **đứng yên khi đo** nên dữ liệu IMU sạch, không bị rung động cơ làm sai. Có **timeout 12 giây** để báo lỗi nếu kẹt.

Tối ưu node thẳng hàng: nếu node kế tiếp gần như cùng hướng (lệch $< 10^\circ$), xe **không dừng** mà đi thẳng liên tục qua nhiều node — giảm số lần dừng/xoay, chạy nhanh và mượt hơn.

3.6.4 Lưu đồ chương trình con đi thẳng (heading correction)

Khi đi thẳng, xe đếm xung encoder để biết đã đi được bao xa, đồng thời **giữ hướng bằng IMU** (không dùng odometry để tránh vòng lặp phản hồi sai). Hai cơ chế hiệu chỉnh chạy song song: hiệu chỉnh hướng (theo IMU) và cân bằng hai bánh (theo encoder).

Hình 3.14 — Lưu đồ chương trình con đi thẳng

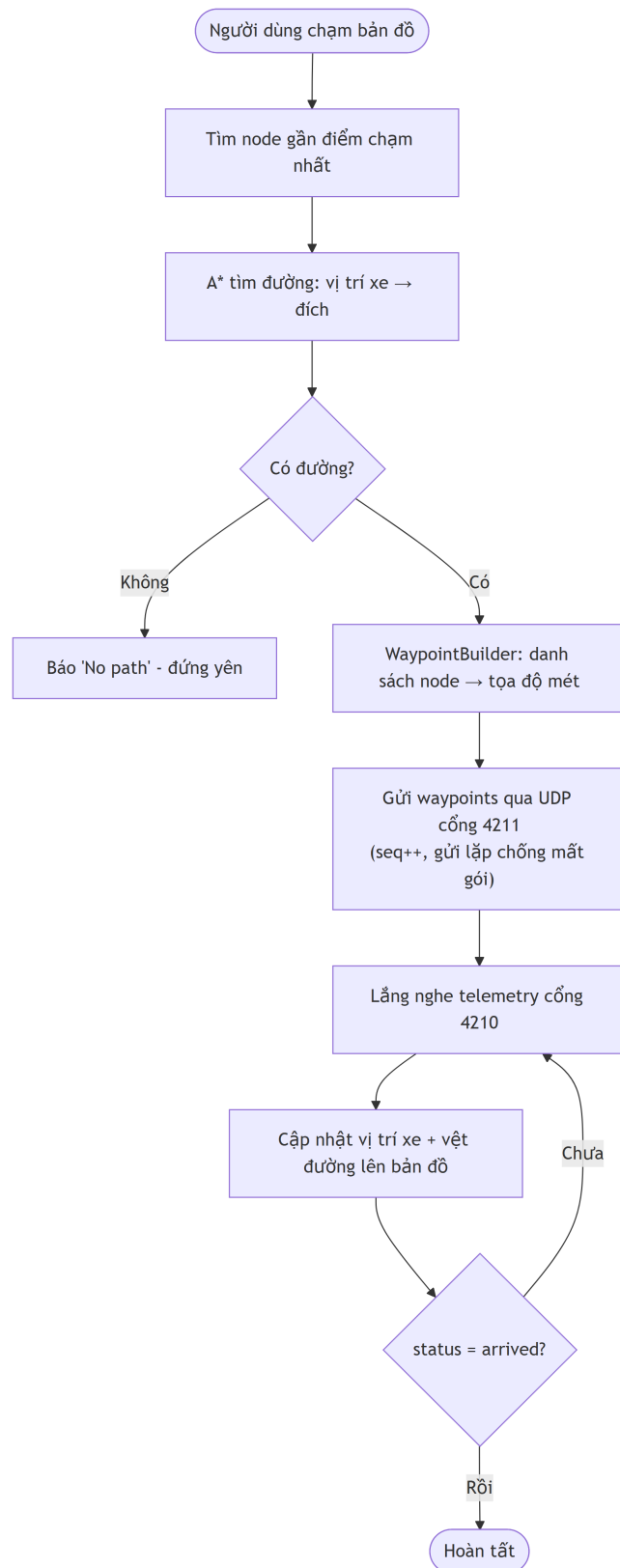


Công thức 9 (số xung mục tiêu): $N_{pulse} = round(dist / 0,5\ m) \times 49$

Công thức 10 (hiệu chỉnh hướng): $c_{heading} = clamp[(\theta_{đích} - \theta_{imu}) \times 175, \pm 50]$

3.6.5 Lưu đồ luồng điều hướng trên ứng dụng (A*)

Hình 3.15 — Lưu đồ luồng điều hướng trên app



Cơ chế tự hiệu chuẩn (lồng trong quá trình đi thẳng) — Công thức 11: sau mỗi đoạn thẳng, firmware so sánh số xung hai bánh để cập nhật hệ số bù động cơ

(motor trim) với tốc độ học **5%/đoạn**, đồng thời ước lượng lại tỉ lệ encoder (trung bình trượt 0,94/0,06). Các hệ số được kẹp trong $[0,72 - 1,28]$ và lưu vào **NVS flash** mỗi 15 giây, giữ lại qua các lần khởi động.

**trim_trái += error × 0,05; trim_phải -= error × 0,05; error = (xung_phải –
xung_trái) / xung_trung_bình**

3.6.6 Lưu đồ giải thuật chế độ tự hành né vật cản

Chế độ tự hành (khởi `autonomous_explorer.cpp`) là một **máy trạng thái** chạy lồng trong vòng lặp 50 Hz khi người dùng bấm **Start** trên ứng dụng. Khối này điều phối ba thành phần: cảm biến HC-SR04 (đo khoảng cách), servo (quét hai bên) và Node Navigator (mượn lại cơ chế xoay 3 tầng đã có để quay $90^\circ/180^\circ$).

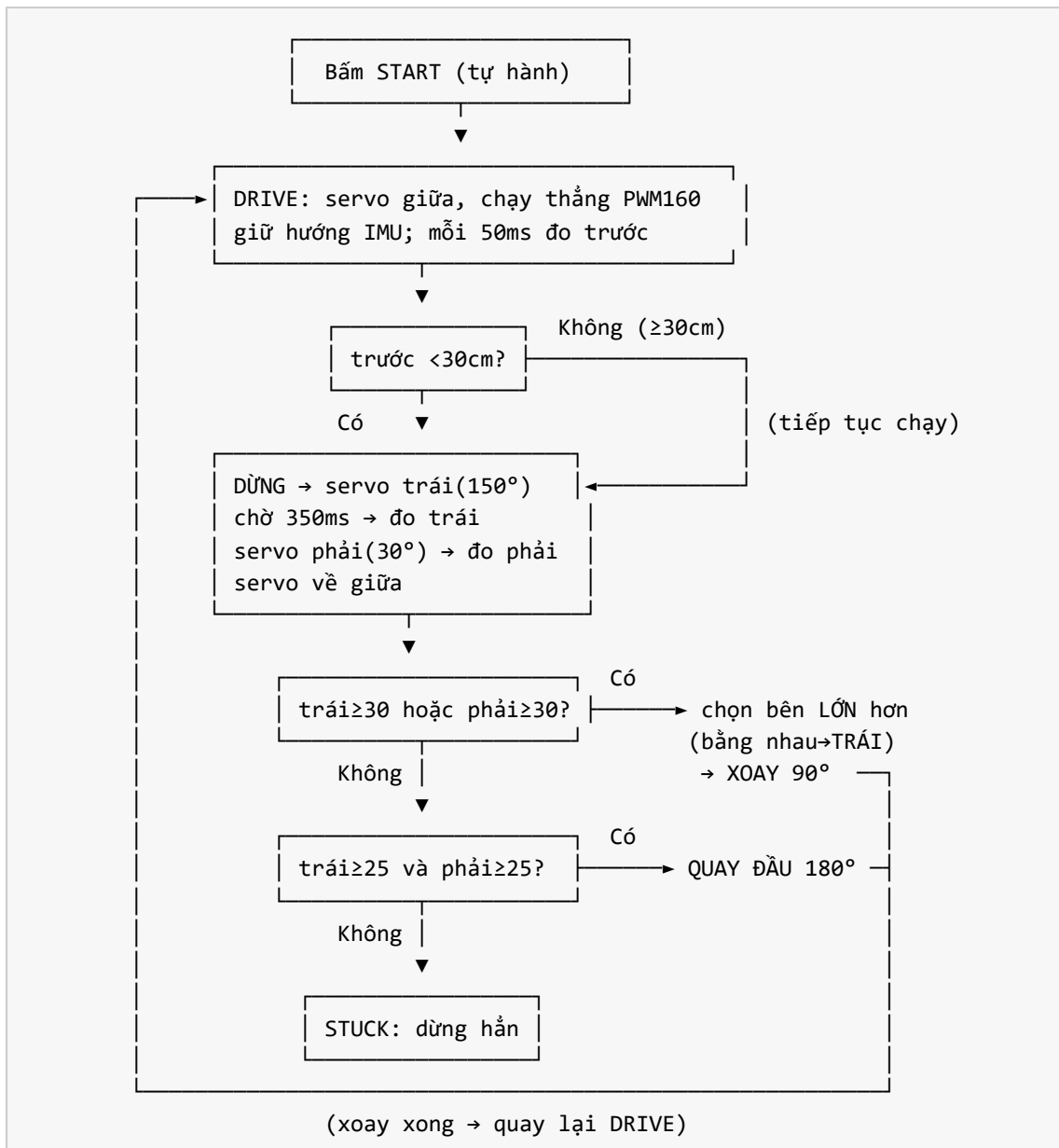
Năm trạng thái và luồng chuyển:

- **Drive (Chạy thẳng):** servo ở giữa (90°), hai bánh chạy tới (PWM 160) đồng thời **giữ hướng bằng IMU** (mượn công thức hiệu chỉnh hướng của chế độ bám lộ trình). Mỗi ~ 50 ms đọc nhanh khoảng cách phía trước. Nếu **< 30 cm** (đọc thêm 1 lần xác nhận để khử nhiễu) → dừng, chuyển sang quét.
- **ScanLeft (Quét trái):** quay servo sang 150° , chờ 350 ms cho ổn định rồi đo kỹ (trung vị 5 mẫu) → lưu khoảng cách trái.
- **ScanRight (Quét phải):** quay servo sang 30° , chờ 350 ms, đo kỹ → lưu khoảng cách phải, đưa servo về giữa rồi **ra quyết định**.
- **Turn (Né):** giao mục tiêu hướng cho Node Navigator (`setStepTurn`) để xoay 90° hoặc 180° ; xong thì đồng bộ lại hướng IMU và quay về Drive.
- **Stuck (Kẹt):** cả ba phía đều hẹp → dừng hẳn, báo trạng thái `stuck` cho tới khi người dùng bấm Stop/Start lại.

Quy tắc quyết định sau khi quét (đúng yêu cầu đề ra):

1. Nếu ít nhất một bên ≥ 30 cm → chọn bên có khoảng cách lớn hơn; nếu bằng nhau thì ưu tiên rẽ trái. Rẽ 90° về bên đó rồi chạy thẳng tiếp.
2. Nếu cả hai bên < 30 cm nhưng đều ≥ 25 cm → quay đầu 180° (đủ chỗ xoay) rồi chạy thẳng.
3. Nếu một trong hai bên < 25 cm (không đủ chỗ xoay) → dừng hẳn (`stuck`).

Hình 3.17 — Lưu đồ giải thuật chế độ tự hành né vật cản



Ghi chú kỹ thuật:

- Khi đứng yên quét, xe dùng phép **đọc kỹ (trung vị 5 mẫu)** cho chính xác; khi chạy dùng **đọc nhanh** để vòng lặp không bị nghẽn (siêu âm có thể chặn tới 30 ms mỗi lần đo khi không có vật phản hồi).
- Pha **Turn** tái sử dụng toàn bộ cơ chế xoay 3 tầng (xoay nhanh + nudge) của Node Navigator nên góc $90^\circ/180^\circ$ đạt độ chính xác như khi bám lộ trình; sau khi xoay, hướng IMU được đồng bộ lại để các đoạn chạy thẳng kế tiếp không tích lũy sai số.
- Quy ước hướng: rẽ trái = $+90^\circ$ (quay ngược chiều kim đồng hồ, yaw tăng), rẽ phải = -90° ; có thể đảo dấu bằng tham số nếu cách lắp cơ khí ngược

chiều.

CHƯƠNG 4: THI CÔNG HỆ THỐNG

4.1 DANH SÁCH LINH KIỆN

Bảng 4.1 — Bảng liệt kê các linh kiện của hệ thống

STT	Tên linh kiện	Số lượng
1	Board ESP32 NodeMCU (ESP32-WROOM-32, 38 chân)	1
2	Module mạch lái động cơ L298N (cầu H đôi)	1
3	Động cơ DC giảm tốc (TT motor vàng) + bánh xe	2
4	Encoder quang LM393 + đĩa mã hóa 20 khe	2
5	Cảm biến quán tính MPU-6050 (6 trục)	1
6	Cảm biến siêu âm HC-SR04	1
7	Servo SG90 (180°) quét cảm biến	1
8	Module còi buzzer	1
9	Pin Li-ion 18650 + đế pin (hoặc pin 7.4V)	2
10	Khung xe robot 2 bánh + bánh đa hướng (caster)	1
11	Dây nối, jumper, header, ốc vít, ke gắn servo	—

4.2 THI CÔNG PHẦN CỨNG VÀ NỐI DÂY

Quá trình thi công phần cứng gồm các bước:

1. **Lắp khung xe:** gắn hai động cơ DC giảm tốc và bánh chủ động vào khung, gắn bánh đa hướng đỡ phía sau, lắp đĩa encoder vào trục bánh.
2. **Gắn mạch:** cố định ESP32, L298N và đế pin lên khung; bố trí MPU-6050 ở gần tâm xe (giảm sai số gyro do bán kính); gắn **servo SG90 ở mũi xe** và lắp **HC-SR04 lên cánh tay servo** để cảm biến quay theo servo, hướng mặc định nhìn thẳng về phía trước.
3. **Đi dây theo Bảng 3.1:** 6 dây điều khiển ESP32 → L298N; ngõ ra động lực L298N → 2 động cơ; 2 dây tín hiệu encoder → GPIO34/32; I2C MPU-6050 → GPIO21/22; Trig/Echo → GPIO5/18; tín hiệu servo → GPIO19; buzzer → GPIO4. **Nối chung GND** toàn hệ thống.

4. **Cấp nguồn:** pin → động lực L298N; 5V out L298N → VIN ESP32; servo lấy 5V (nên có tụ lọc gần servo để tránh sụt áp gây reset khi servo khởi động).

Lưu ý kỹ thuật: chân GPIO34/32 chỉ là input (không kéo nội bộ) nên dùng điện trở kéo của module encoder; đặt MPU-6050 phẳng, tránh rung; dây encoder và dây động cơ đi tách nhau để giảm nhiễu EMI; cụm servo + HC-SR04 đặt cân bằng ở mũi xe, dây tín hiệu servo đủ chùng để không cản cánh tay quay.

[CHÈN HÌNH 4.1 — Ảnh xe robot sau khi lắp ráp hoàn chỉnh]

4.3 LẬP TRÌNH FIRMWARE VÀ ỨNG DỤNG

Firmware (PlatformIO): mã nguồn được tổ chức module hóa theo thư mục `communication / controllers / sensors / localization / calibration`, biên dịch và nạp bằng PlatformIO:

```
cd firmware
pio run                # biên dịch
pio run --target upload # nạp xuống ESP32
pio device monitor     # giám sát Serial
```

Có môi trường `l298n_test` riêng để kiểm tra mạch lái động cơ độc lập trước khi tích hợp.

Ứng dụng (Flutter):

```
cd mobile
flutter pub get
flutter test                # chạy unit test
flutter run                 # chạy với UDP thật (cần ESP32)
flutter run --dart-define=USE_SIMULATOR=true # chạy với giả lập
```

Kiểm thử đơn vị: các module thuật toán quan trọng được kiểm thử tự động: A* (đường ngắn nhất, không có đường, cạnh bị chặn), nạp bản đồ, bộ dự đoán waypoint, bộ phân tích telemetry, và toán odometry — chạy được mà không cần phần cứng.

[CHÈN HÌNH 4.2 — Ảnh màn hình bản đồ khi xe đang di chuyển]

CHƯƠNG 5: KẾT QUẢ - NHẬN XÉT - ĐÁNH GIÁ

5.1 KẾT QUẢ LÝ THUYẾT

Sau khi thực hiện đề tài, về mặt lý thuyết đã đạt được:

- Nắm vững cách sử dụng ESP32 và các chuẩn giao tiếp **GPIO, I2C, PWM/LEDC, Trigger-Echo, ngắt ngoài và WiFi/UDP**; biết tra cứu và áp dụng datasheet (MPU-6050, L298N, HC-SR04).
- Hiểu và cài đặt được bài toán **odometry vi sai** và **bộ lọc bù** kết hợp encoder + IMU; hiểu bản chất sai số dead-reckoning và cách giảm thiểu.
- Cài đặt thành công thuật toán **A*** với heuristic Euclid và chi phí phụ phạt rẽ; hiểu khái niệm heuristic admissible.
- Thiết kế được **giao thức UDP** thời gian thực với cơ chế chống trùng, chia mảnh và telemetry.
- Cài đặt được **máy trạng thái né vật cản phản ứng (reactive obstacle avoidance)** dựa trên cảm biến siêu âm + servo quét, và điều khiển servo bằng LEDC thô (PWM 50 Hz, 16-bit) — hiểu cách phối hợp nhiều thiết bị trong một vòng lặp thời gian thực mà không nghẽn.
- Biết tổ chức một dự án phần mềm **module hóa** (firmware C++ + app Flutter) kèm kiểm thử đơn vị.

5.2 KẾT QUẢ THỰC HÀNH

Đã hoàn thiện một xe robot tự hành chạy được end-to-end: từ thao tác **chạm chọn đích trên app**, xe **tự tính đường, tự xoay và bám lộ trình** qua các node tới đích, với **vị trí hiển thị thời gian thực** trên bản đồ. Đã thực hiện được quá trình lắp ráp cơ khí, đi dây, nạp firmware, build app, và **tinh chỉnh thực nghiệm** các tham số (số xung/node, hệ số hiệu chỉnh hướng, PWM xoay, thời gian ổn định nudge). Cơ chế **tự hiệu chuẩn** giúp xe đi thẳng cân bằng hơn sau vài đoạn. Chế độ **giả lập** cho phép demo và phát triển app độc lập với phần cứng.

Ngoài ra đã triển khai và chạy được **chế độ tự hành né vật cản**: xe tự chạy thẳng, gặp vật cản dưới 30 cm thì dừng, quay servo quét hai bên và tự chọn hướng (rẽ 90° về bên thoáng, quay đầu 180° khi cả hai bên bị chặn nhưng đủ chỗ xoay, hoặc dừng khi kẹt). Người dùng bật/tắt chế độ này bằng nút Start/Stop trên

màn hình Tự hành; các trạng thái `exploring/scanning/avoiding/stuck` và khoảng cách vật cản được phản hồi trực tiếp lên app.

Để đánh giá định lượng các chức năng trên, nhóm đã tiến hành đo đạc thực nghiệm với kết quả chi tiết trình bày dưới đây.

Ghi chú: các ô "... " trong bảng là số đo thực tế cần điền sau khi chạy thử robot; các cột giá trị lệnh và điều kiện đã được cố định sẵn. Mỗi phép đo nên lặp ít nhất 5–10 lần để lấy trung bình.

5.2.1 Điều kiện và phương pháp thực nghiệm

Các thí nghiệm được thực hiện trong nhà, trên sàn phẳng (gạch men/xi măng), pin sạc đầy, nguồn IMU và servo tách riêng qua module giảm áp để giảm nhiễu. Dụng cụ đo gồm: thước cuộn (vạch mm) đo khoảng cách, thước đo độ hoặc vòng tròn chia độ in trên giấy đo góc, đồng hồ bấm giây đo thời gian. Số liệu định vị được thu qua telemetry UDP (10 Hz) hoặc Serial Monitor (chu kỳ 500 ms). Các thông số phần cứng liên quan: đường kính bánh 6.5 cm, 20 xung/vòng encoder, khoảng cách hai bánh $b = 10.5$ cm, một node = 0.5 m ≈ 49 xung (Công thức 9).

5.2.2 Độ chính xác đo quãng đường (odometry)

Phương pháp: Ra lệnh cho xe đi thẳng các quãng 0.5 m, 1.0 m, 2.0 m; đo quãng đường thực tế bằng thước, đồng thời ghi quãng đường do odometry báo về (Công thức 1). Sai số tương đối tính bằng $| \text{quãng thực} - \text{quãng lệnh} | / \text{quãng lệnh} \times 100\%$.

Bảng 5.1 — Kết quả đo độ chính xác quãng đường khi đi thẳng

Quãng đường lệnh (m)	Odometry báo về (m)	Đo thực tế TB (m)	Sai số TB (cm)	Sai số (%)
0.5	...	...	...	...
1.0	...	...	...	...
2.0	...	...	...	...

Nhận xét: Sai số quãng đường chủ yếu do bánh xe nén/trượt khiến chu vi thực nhỏ hơn danh nghĩa. Hằng số số xung mỗi node lấy theo lý thuyết (≈ 49 xung cho 0,5 m, tự suy từ `CM_PER_PULSE`) và được kiểm chứng bằng đo thực tế; sai số tương đối thường giảm khi quãng đường tăng do thành phần sai số cố định (khởi

động/dừng) ít ảnh hưởng hơn. (Điền nhận xét định lượng theo số đo: sai số trung bình ...%.)

5.2.3 Độ chính xác xoay góc (IMU)

Phương pháp: Đặt xe tại tâm vòng tròn chia độ, ra lệnh xoay 45°, 90°, 180°; đọc góc thực tế đạt được. Cơ chế xoay gồm pha *xoay nhanh theo wóc lượng thời gian* cho góc lớn, sau đó pha *nudge* (xoay từng nhịp ngắn, đo IMU, lặp) để tinh chỉnh tới khi sai số $\leq$ ngưỡng cho phép 4°.

Bảng 5.2 — Kết quả đo độ chính xác xoay góc

Góc lệnh (°)	Góc đo TB (°)	Sai số TB (°)	Độ lệch chuẩn (°)
45	...	...	...
90	...	...	...
180	...	...	...

Nhận xét: Việc tách nguồn riêng cho IMU giúp dữ liệu góc sạch hơn, giảm dao động ở pha nudge. Góc 180° (quay đầu) thường có sai số lớn nhất do quãng xoay dài, quán tính và trượt bánh nhiều hơn. (Điền: sai số trung bình ...°, có/không nằm trong dung sai 4°.)

5.2.4 Khả năng giữ hướng khi đi thẳng

Phương pháp: Cho xe đi thẳng 3 m dọc một đường kẻ sẵn; đo độ lệch ngang của xe so với đường chuẩn tại đích và quy đổi ra góc lệch.

Bảng 5.3 — Kết quả đánh giá khả năng giữ hướng

Lần đo	Độ lệch ngang tại 3 m (cm)	Góc lệch tương đương (°)
1	...	...
2	...	...
3	...	...
Trung bình	...	...

Nhận xét: Nhờ vòng giữ hướng kết hợp IMU và cân bằng encoder (Công thức 10, 11), xe bám đường thẳng tương đối tốt. (Điền: độ lệch ngang trung bình ...cm)

trên 3 m, tương đương ...°.)

5.2.5 Độ chính xác điều hướng theo lưới điểm (node)

Phương pháp: Nạp lộ trình gồm nhiều node, cho xe chạy hết lộ trình rồi đo sai số vị trí và sai số góc tại điểm đích so với vị trí lý thuyết.

Bảng 5.4 — Kết quả điều hướng theo lộ trình node

Lộ trình	Số node	Tổng quãng (m)	Sai số vị trí cuối (cm)	Sai số góc cuối (°)
Đường thẳng 4 node	4	2.0	...	...
Hình vuông cạnh 0.5 m	4	2.0	...	...
Chữ L	...	...	...	...

Nhận xét: Sai số tích lũy tăng theo số lần xoay và tổng quãng đường — đặc trưng của định vị dead-reckoning không có mốc tham chiếu tuyệt đối. (Điện: với lộ trình ..., sai số vị trí cuối ...cm.)

5.2.6 Độ chính xác cảm biến siêu âm kết hợp servo quét

Phương pháp: Đặt vật cản tại các khoảng cách biết trước (10, 20, 30, 50 cm) ở ba hướng quét của servo — trái (+90°), giữa (0°), phải (−90°) — và so sánh giá trị HC-SR04 đo được với khoảng cách thực (Công thức 2). (Sau khi sửa dải xung servo về 0.5–2.5 ms, servo quét đủ ±90° mỗi bên thay vì chỉ ~±45° như trước, nên phép đo hai bên phản ánh đúng khoảng trống trái/phải.)

Bảng 5.5 — Kết quả đo khoảng cách bằng HC-SR04 theo ba hướng quét

Khoảng cách thực (cm)	Đo hướng giữa (cm)	Đo hướng trái +90° (cm)	Đo hướng phải −90° (cm)	Sai số TB (cm)
10	...	...	...	...
20	...	...	...	...
30	...	...	...	...
50	...	...	...	...

Nhận xét: Độ chính xác này quyết định độ tin cậy của ngưỡng dừng 30 cm và ngưỡng quay đầu 25 cm trong chế độ tự hành. (Điền: sai số đo trung bình ...cm trong tầm 10–50 cm.)

5.2.7 Khả năng né vật cản ở chế độ tự hành

Phương pháp: Cho xe chạy tự hành trong môi trường có vật cản đặt ngẫu nhiên, thực hiện nhiều lượt thử; ghi nhận khoảng cách dừng trước vật cản và việc né có thành công (không va chạm) hay không.

Bảng 5.6 — Kết quả thử nghiệm né vật cản tự hành

Lượt thử	Khoảng cách dừng (cm)	Né thành công (Có/Không)	Ghi chú
1	...	...	...
2	...	...	...
3	...	...	...
4	...	...	...
5	...	...	...

Tỉ lệ né thành công: ... / ... lượt (...%); khoảng cách dừng trung bình: ...cm.

Nhận xét: Các trường hợp thất bại (nếu có) thường do vật cản mỏng/hấp thụ sóng siêu âm hoặc đặt lệch ngoài ba hướng quét (điểm mù). (Điền: tỉ lệ thành công ...%.)

5.2.8 Độ trôi góc hướng của IMU (drift)

Phương pháp: Để xe đứng yên hoàn toàn sau khi hiệu chỉnh bias, ghi giá trị góc yaw tại các mốc thời gian để đánh giá độ trôi tích lũy.

Bảng 5.7 — Kết quả đo độ trôi góc yaw khi đứng yên

Thời gian đứng yên (s)	Độ trôi yaw (°)
30	...
60	...
300	...

Nhận xét: Nhờ hiệu chỉnh bias lúc khởi động và vùng chết vận tốc góc (deadband 0.01 rad/s), độ trôi được hạn chế ở mức chấp nhận được trong thời gian một lộ trình điển hình. (*Điễn: độ trôi ...° sau 5 phút.*)

5.3 NHẬN XÉT VÀ ĐÁNH GIÁ

Về cơ bản hệ thống hoạt động đầy đủ các chức năng chính. Khối điều khiển, định vị và truyền thông hoạt động ổn định. Hạn chế lớn nhất nằm ở **sai số định vị tích lũy (dead-reckoning)**: do chỉ dựa vào odometry tương đối, sau quãng đường dài hoặc nhiều lần xoay, vị trí ước lượng lệch dần so với thực tế (trượt bánh, sai số gyro) — đây là hạn chế cố hữu của phương pháp không có mốc định vị tuyệt đối. Cơ chế nudge step-stop-measure cho góc xoay chính xác nhưng làm thời gian xoay lâu hơn so với xoay liên tục.

Bảng 5.8 — Bảng mô tả trạng thái hoạt động của các khối

STT	Tên khối	Đánh giá hoạt động
1	Khối xử lý trung tâm (ESP32)	Rất tốt
2	Khối động cơ (L298N + DC motor)	Rất tốt
3	Khối encoder (LM393 ×2)	Tốt (nhạy nhiều EMI, đã lọc)
4	Khối cảm biến góc IMU (MPU-6050)	Tốt (có trôi nhẹ khi chạy lâu)
5	Khối định vị odometry (lọc bù)	Khá (sai số tích lũy theo quãng đường)
6	Khối điều hướng A* + Node Navigator	Tốt
7	Khối truyền thông WiFi/UDP	Rất tốt
8	Khối cảm biến khoảng cách (HC-SR04)	Tốt (dùng trong chế độ tự hành; đôi khi nhiễu ở góc/bề mặt hút âm)
9	Khối servo quét + chế độ tự hành	Tốt (né được vật cản; phụ thuộc độ chính xác siêu âm)
10	Ứng dụng di động (Flutter)	Rất tốt

Hệ thống được thiết kế theo mô hình học thuật nên về cơ bản chưa đầy đủ chức năng cao cấp và còn hạn chế: chưa có định vị tuyệt đối (chỉ odometry tương đối); chế độ né vật cản hiện là **phản ứng cục bộ (reactive)** — tránh được vật ngay trước mặt nhưng chưa lập lại đường đi toàn cục theo bản đồ; siêu âm chỉ "thấy" theo ba hướng quét nên còn điểm mù; bản đồ phải biết trước, chưa tự xây bản đồ. Mức độ áp dụng thực tế còn ở quy mô phòng thí nghiệm, cần mở rộng thêm để đưa vào ứng dụng thực tế.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 KẾT LUẬN

Đề tài đã thiết kế và xây dựng thành công một **xe robot tự hành** hoàn chỉnh, chạy được toàn trình từ chọn đích đến bám lộ trình. Về cơ bản hệ thống đáp ứng đủ các chức năng:

- Người dùng chọn đích trên app, app tự **tìm đường ngắn nhất bằng A***.
- Xe **tự định vị bằng odometry** (encoder + IMU qua bộ lọc bù) và **bám theo lộ trình** với cơ chế xoay 3 tầng và giữ hướng khi đi thẳng.
- **Hiển thị vị trí thời gian thực** trên bản đồ qua telemetry UDP 10 Hz.
- Có **chế độ tự hành né vật cản** dùng HC-SR04 + servo quét để tự dừng, dò hai bên và chọn hướng đi.
- Hỗ trợ **điều khiển tay, tự hiệu chuẩn** và **chế độ giả lập** để kiểm thử.

Hệ thống có kiến trúc module hóa rõ ràng, chi phí thấp, hoạt động độc lập không cần Internet — là nền tảng tốt để học tập và phát triển tiếp.

6.2 HƯỚNG PHÁT TRIỂN

Để nâng cao độ chính xác và khả năng ứng dụng thực tế, đề tài có thể phát triển theo các hướng:

- **Bổ sung định vị tuyệt đối** để khắc phục sai số tích lũy: thêm cảm biến mốc (line/landmark), camera nhận diện thẻ ArUco, hoặc UWB anchor.
- **Nâng cấp né vật cản từ phản ứng lên có bản đồ**: kết hợp chế độ tự hành hiện có với bản đồ — khi gặp vật cản thì **chặn cạnh đường tương ứng và cho A* tự tính lại đường (replan)** thay vì chỉ tránh cục bộ; thay/thêm cảm biến ToF/LiDAR mini hoặc nhiều siêu âm để giảm điểm mù và quét nhanh hơn (thay cho việc xoay servo từng bên).
- **Nâng cấp định vị bằng SLAM** để xe tự xây bản đồ thay vì dùng bản đồ biết trước.
- **Cải tiến điều khiển**: áp dụng bộ điều khiển PID/đóng vòng vận tốc bằng encoder cho động cơ; xoay liên tục có phản hồi thay cho nudge để rút ngắn thời gian.

- **Mở rộng hệ thống IoT:** đưa dữ liệu lên đám mây, điều khiển/giám sát từ xa, phối hợp **nhiều robot** thành một mạng lưới.
-
-

TÀI LIỆU THAM KHẢO

Datasheet và tài liệu kỹ thuật:

- [1] Espressif Systems, *ESP32 Series Datasheet*, 2024. [Online]. https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [2] Espressif Systems, *ESP32-WROOM-32 Datasheet*, 2024. [Online]. https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf
- [3] InvenSense (TDK), *MPU-6000/MPU-6050 Product Specification & Register Map*, Rev. 3.4/4.2. [Online]. <https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6050/>
- [4] STMicroelectronics, *L298 Dual Full-Bridge Driver Datasheet*. [Online]. <http://www.st.com/resource/en/datasheet/l298.pdf>
- [5] SparkFun Electronics, *HC-SR04 Ultrasonic Sensor Module*. [Online]. <https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>

Tài liệu thuật toán và nền tảng:

- [6] P. E. Hart, N. J. Nilsson, B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths", *IEEE Trans. on Systems Science and Cybernetics*, 1968.
 - [7] S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics*, MIT Press, 2005 (chương về odometry và mô hình chuyển động vi sai).
 - [8] Espressif Systems, *ESP-IDF / Arduino-ESP32 Programming Guide* (LEDC, GPIO interrupts, WiFi SoftAP). [Online]. <https://docs.espressif.com/>
 - [9] Google, *Flutter Documentation và Dart Language Tour*. [Online]. <https://docs.flutter.dev/>
-
-

PHỤ LỤC

Mã nguồn hệ thống (firmware ESP32 + ứng dụng Flutter): <https://github.com/ngxziet/doantotnghiep.git>

Cấu trúc mã nguồn:

- `firmware/src/` — firmware ESP32 (PlatformIO): `communication/` (UDP), `controllers/` (motor, node navigator, **autonomous explorer**), `sensors/` (encoder, MPU-6050, HC-SR04, **servo quét**), `localization/` (odometry), `calibration/` (tự hiệu chuẩn), `config.h`, `main.cpp`.
- `mobile/lib/` — ứng dụng Flutter: `models/`, `planning/` (A*, road map, waypoint builder), `services/` (UDP, simulator), `screens/` (4 màn hình: Map, Test, Manual, **Tự hành**).
- `map_data/environment.json` — bản đồ lưới waypoint 5×5 m.
- `docs/` — tài liệu (codebase summary, hướng dẫn thuật toán firmware).

Thông số mạng ESP32: SSID `RobotCar` · Mật khẩu `12345678` · IP `192.168.4.1` · Cổng telemetry `4210` · Cổng lệnh `4211`.

GHI CHÚ KHI ĐÓNG QUYỀN (XÓA TRƯỚC KHI NỘP)

*Báo cáo này được sinh tự động từ mã nguồn thực tế của dự án. Trước khi nộp cần bổ sung **ảnh thật** vào các vị trí đánh dấu [CHÈN HÌNH ...]:*

- Logo trường, ảnh thực tế xe robot, ảnh giao diện 4 màn hình app, ảnh màn hình bản đồ khi chạy.
- Sơ đồ nguyên lý / nối dây vẽ trên Proteus hoặc Fritzing (Hình 3.10) và sơ đồ chân ESP32 (Hình 3.3).
- Các hình minh họa lý thuyết (I2C, PWM, Trigger-Echo, xe vi sai, A*) ở Chương 2.

*Các **lưu đồ (mermaid)** ở mục 3.6 sẽ tự render thành sơ đồ khối khi mở bằng trình xem hỗ trợ Mermaid (VS Code + extension, Typora, GitHub, HackMD...). Nếu xuất sang Word, có thể chụp ảnh các lưu đồ này hoặc vẽ lại bằng công cụ vẽ flowchart.*

Số liệu sai số thực nghiệm (mục 5.2/5.3) hiện mô tả định tính; nên đo và điền số đo thực tế (ví dụ: sai số vị trí trung bình sau quãng đường 5 m, sai số góc sau khi xoay $90^\circ/180^\circ$) để phân đánh giá thuyết phục hơn.